



**TOR VERGATA**  
UNIVERSITÀ DEGLI STUDI DI ROMA

**Macroarea di Ingegneria**  
**Dipartimento di Ingegneria Civile e Ingegneria Informatica**

# **Distributed System Architectures**

**Corso di Sistemi Distribuiti e Cloud Computing**  
**A.A. 2026/27**

**Valeria Cardellini**

**Laurea Magistrale in Ingegneria Informatica**

# Software vs. system architecture of DS

---

- **Software architecture:** logical organization and interaction of software components in a DS
- **System architecture:** final instantiation (including deployment) of a software architecture
  - Software components need to be placed on system resources
  - E.g., placing a containerized microservice on a machine
- Let's first focus on software architectures for DS

# Architectural styles for DS

---

- Architectural style: set of design decisions concerning sw architecture
  - Defined primarily in terms of **components** and **connectors**
- **Component**
  - Modular unit with well-defined interfaces
  - Replaceable within its environment
- **Connector**
  - Mechanism enabling interaction among components, handling communication, coordination or cooperation
  - Example: remote procedure call, messaging system
- Also includes:
  - How components are connected
  - What data is exchanged
  - How components and connectors are configured together

# Main architectural styles for DS

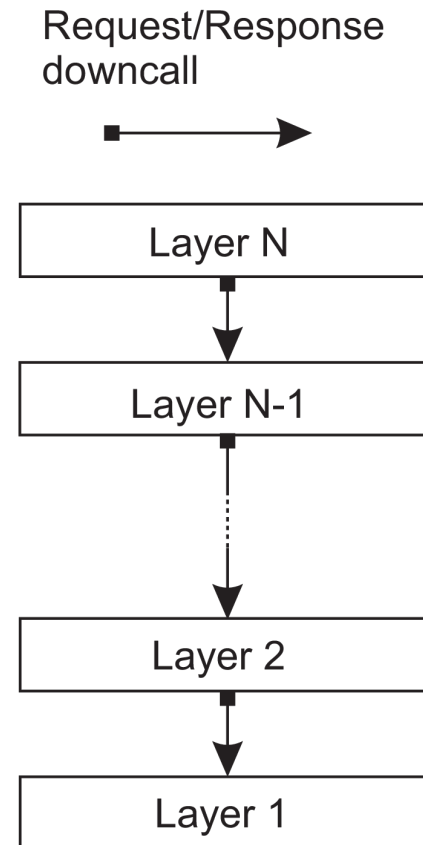
---

- Layered style
- Service-oriented style
  - Object-oriented
  - Microservices
  - RESTful
- Publish-subscribe style

# Layered style

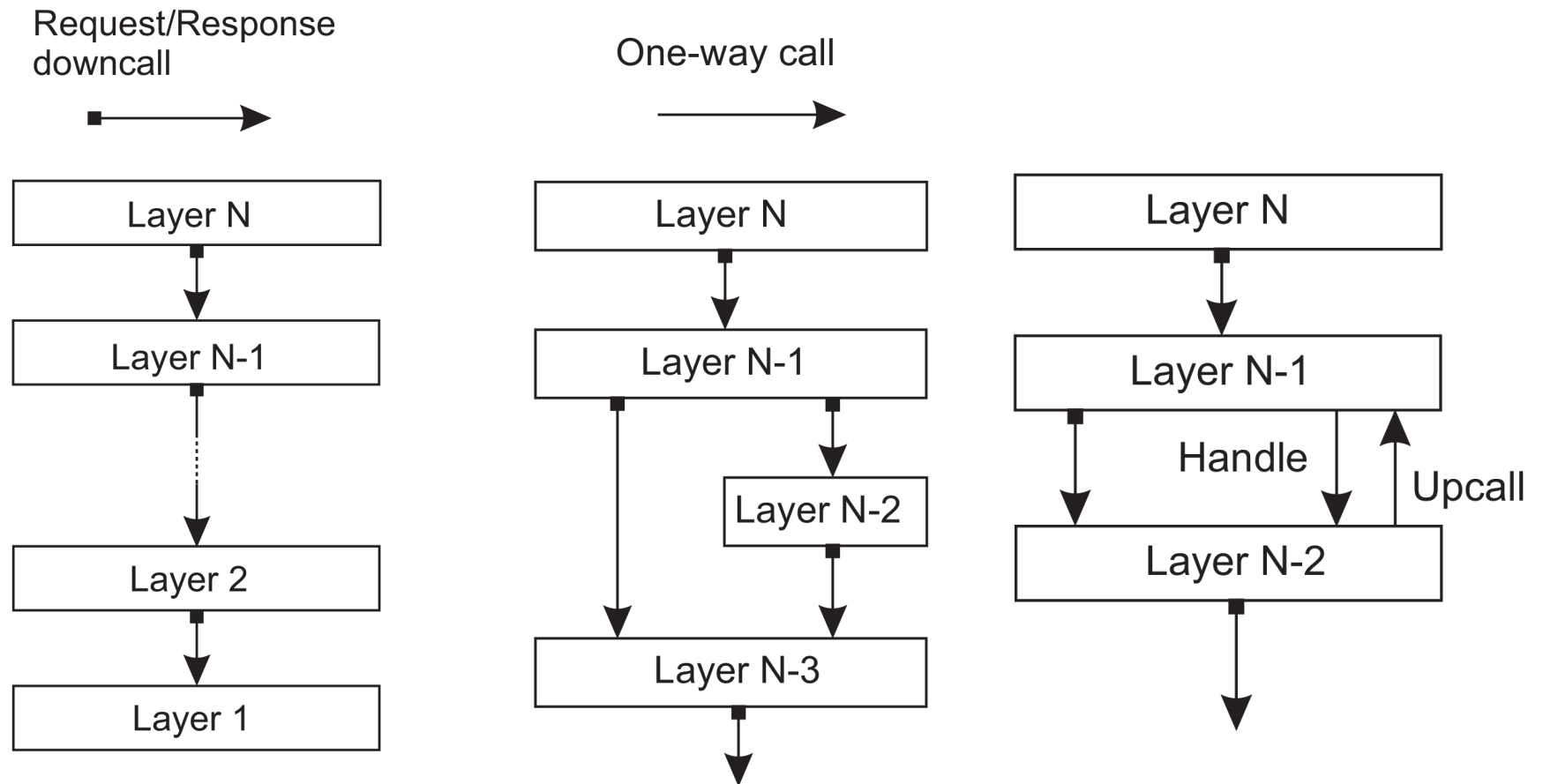
---

- Components are organized in *layers*
- Component at layer  $i$  invokes component at layer  $j$  (*with  $j < i$* )
- Components communicate by message exchange
  - Request/response downcall
- ✓ Separation of concerns among components
  - E.g., web app based on MVC design



# Layered style

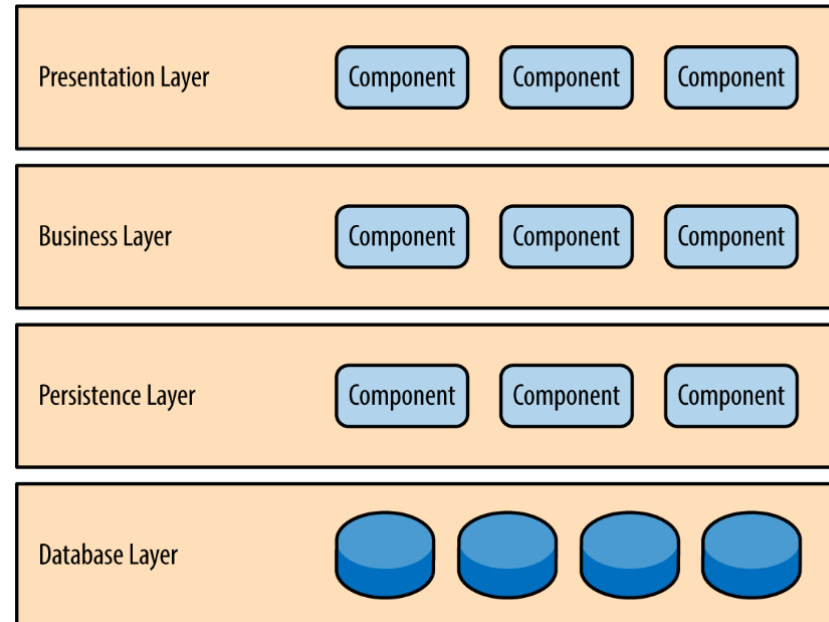
- Different layered organizations



# Application layering

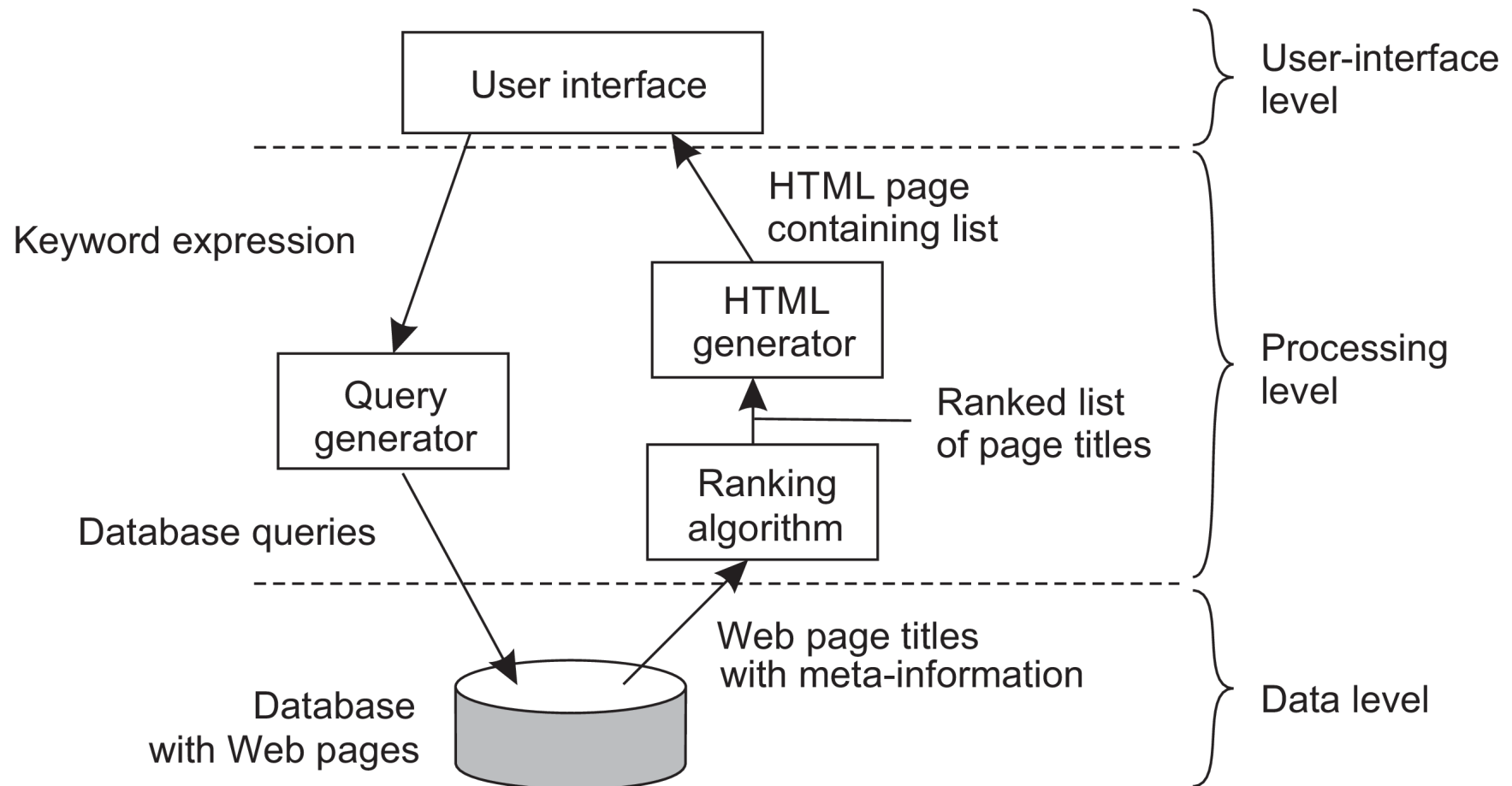
---

- Traditional layered architecture: presentation, business, persistence, database
  - Business and persistence layers can be combined into a single layer
  - Frequently adopted in traditional distributed information systems using relational DB technology



# Application layering: example

- A simple Web search engine



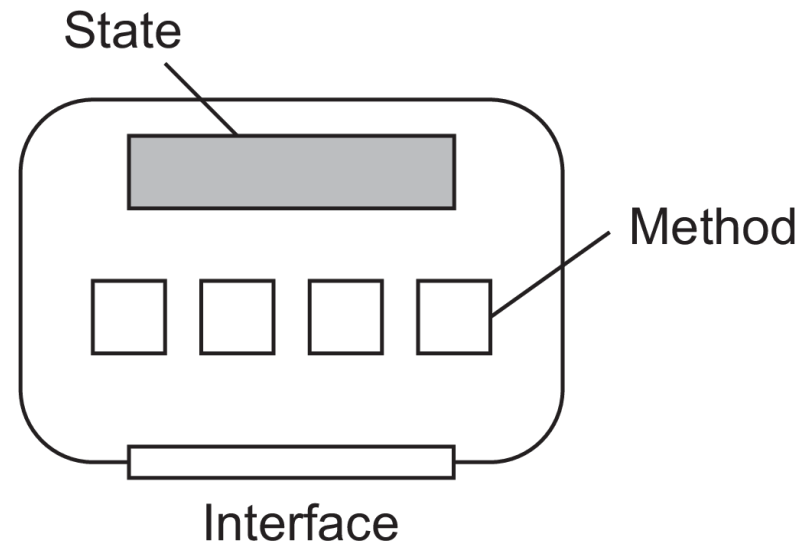
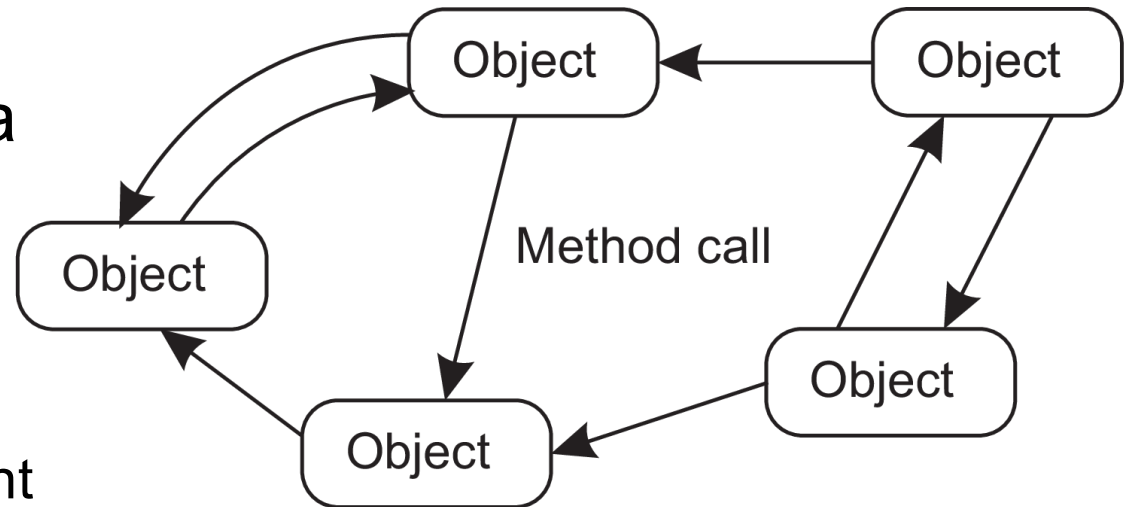
# Service-oriented style

---

- Collection of separate, independent entities
- Each entity encapsulates a service
  - Its logic and state
- Entity = object, microservice, or resource
- Variations
  - Object-based architectural style
  - Microservices architectural style
  - RESTful architectural style

# Object-based style

- Entity = object
- Object encapsulates data and methods
  - ✓ Encapsulation and information hiding reduce management complexity
  - ✓ Reusability across different apps
  - ✓ Wrapping of legacy components
- Communication between objects via RPC
- Early-day technologies: JAVA RMI, CORBA



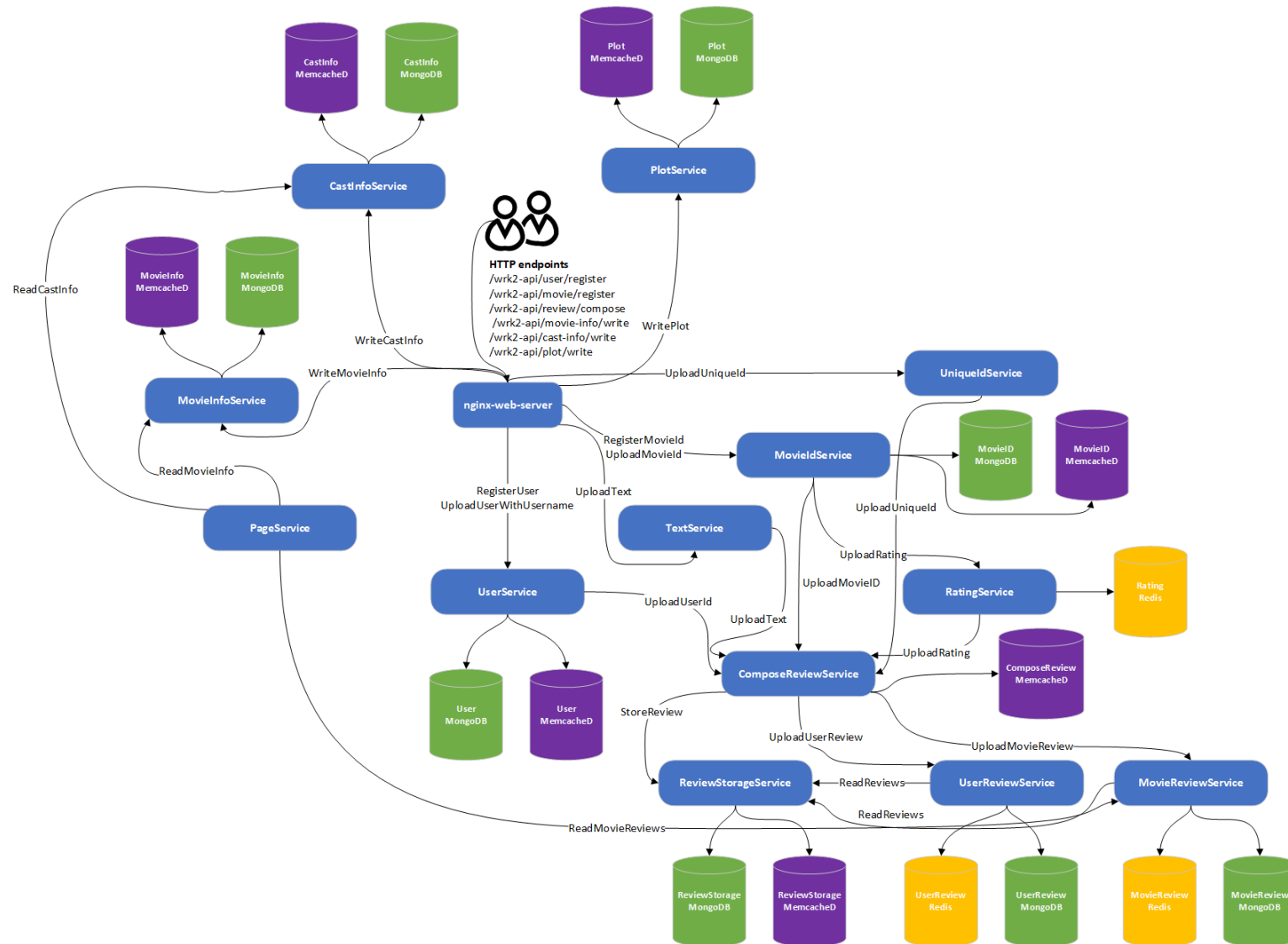
# Microservices style

---

- Entity = microservice
- Applications are structured as a collection of loosely coupled, cooperating services
- Goals: build, manage, and evolve architectures composed of small, self-contained and independently scalable services that communicate over well-defined APIs
  - ✓ Modularization: app decomposed into independently deployable units that can be rapidly deployed and scaled
  - ✓ Fault-tolerance and isolation: failures are contained within individual microservices (e.g., patterns like circuit breaker)
- See upcoming lessons

# Microservices style: example

- A social-media microservice architecture



<https://github.com/delimitrou/DeathStarBench>

# RESTful style

---

- Entity = resource
- Applications structured as a collection of resources managed by components using HTTP methods
- **Representational State Transfer (REST)**
  - Proposed by Roy Fielding (HTTP/1.1 author)
  - Resources added, removed, retrieved, and modified via standard **HTTP methods**: GET, PUT, POST and DELETE

| Operation | Description                                             |
|-----------|---------------------------------------------------------|
| PUT       | Create a new resource                                   |
| GET       | Retrieve the state of a resource in some representation |
| DELETE    | Delete a resource                                       |
| POST      | Modify a resource by transferring a new state           |

- Resources identified through a unique naming scheme (Uniform Resource Identifier, **URI**)  
URI = scheme:[//authority]path[?query][#fragment]  
authority = [userinfo@]host[:port]

# REST operations

---

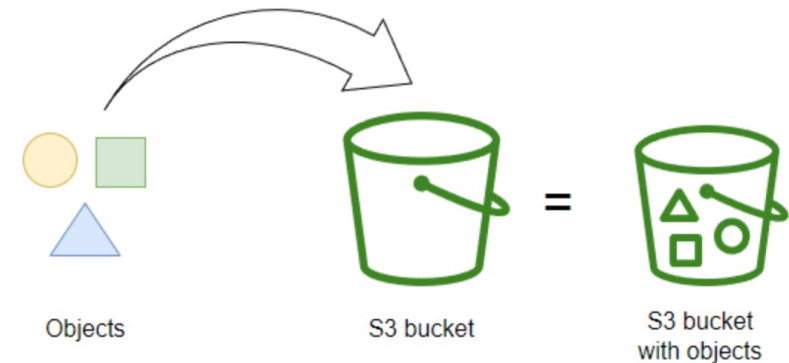
- Components expose a uniform interface
- Messages sent to/from component are self-described
- **Stateless** interaction
  - No server-side sessions, every request is self-contained
  - State representations are transferred between client and server via request/response payloads
- Example: online bookstore
  - Resource: a cart representation (`/carts/12345`) containing books and quantities
  - Server-side components process requests (e.g., `POST /carts/12345/items`) to manipulate the resource state
    - E.g., add another book to cart: `{"bookId": 42, "quantity": 1}`

# Example: S3 REST API

- S3: cloud storage service by AWS, organized as a **key-based** object store
  - **Objects** (files) are stored into **buckets** (top-level containers)

- Resource model

- Files and buckets as distinct resources
- Flat structure: no real directory hierarchy



- Logical hierarchy simulated by using object names with directory structure: `photos/puppy.jpg`

- Resource identification

- Object *objectname* stored in bucket *bucketname* is uniquely referred to by its URI

`https://bucketname.s3.region.amazonaws.com/objectname`

e.g., `https://example-bucket.s3.us-west-2.amazonaws.com/photos/puppy.jpg`

S3 REST API: <https://docs.aws.amazon.com/AmazonS3/latest/API/Welcome.html>

# Example: S3 REST API

---

- Operations via HTTP requests:
  - Create bucket/object: `PUT` (along with its URI)
  - List objects in bucket: `GET` on bucket URI
  - Read object: `GET` on full object URI
- Example: retrieve object from bucket (GetObject)  
`GET /photos/puppy.jpg HTTP/1.1`  
`Host: example-bucket.s3.us-west-2.amazonaws.com`  
`Date: date`  
`Authorization: authorization string`
- Example: add object to bucket (PutObject)  
`PUT /photos/puppy.jpg HTTP/1.1`  
`Host: example-bucket.s3.us-west-2.amazonaws.com`  
`Date: date`  
`Authorization: authorization string`
- HTTP Authorization header: to authenticate S3 request
- Permission for operations (e.g., `WRITE` permission on bucket): managed via AWS IAM roles and permissions

# Decoupling

---

- Strong dependencies between components introduce limitations
- Solution: let components **indirectly communicate** through some intermediary
  - Clean separation between computation and coordination

**“All problems in computer science can be solved by another level of indirection”**

(David Wheeler, Titan project)

- **Decoupling**: enabling factor to
  - Achieve greater flexibility
  - Define architectural styles that allow to better exploit distribution, scalability, and elasticity

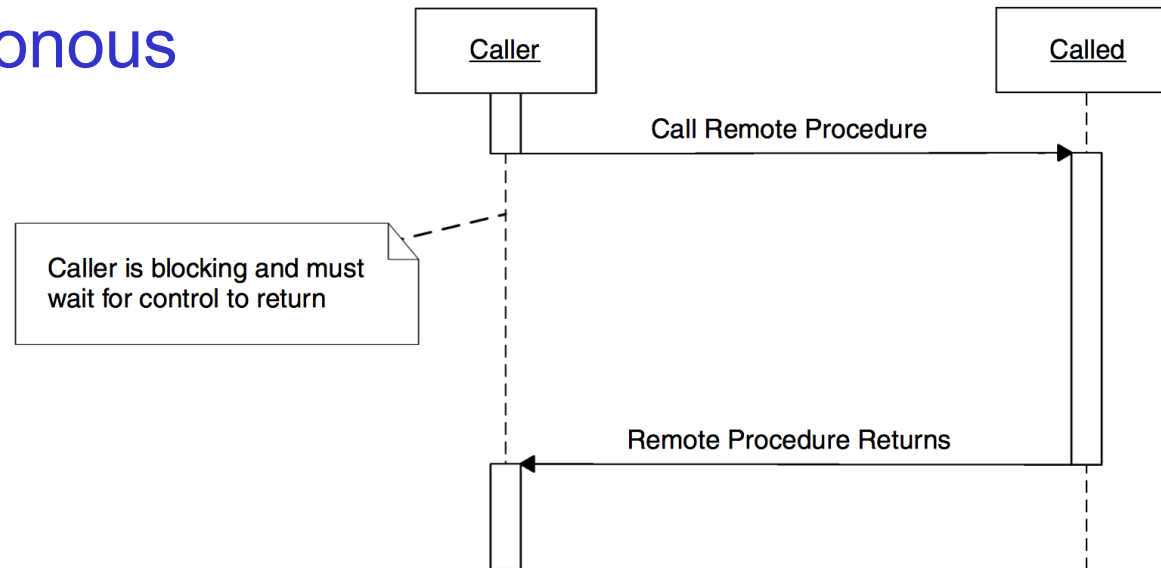
# Decoupling properties

---

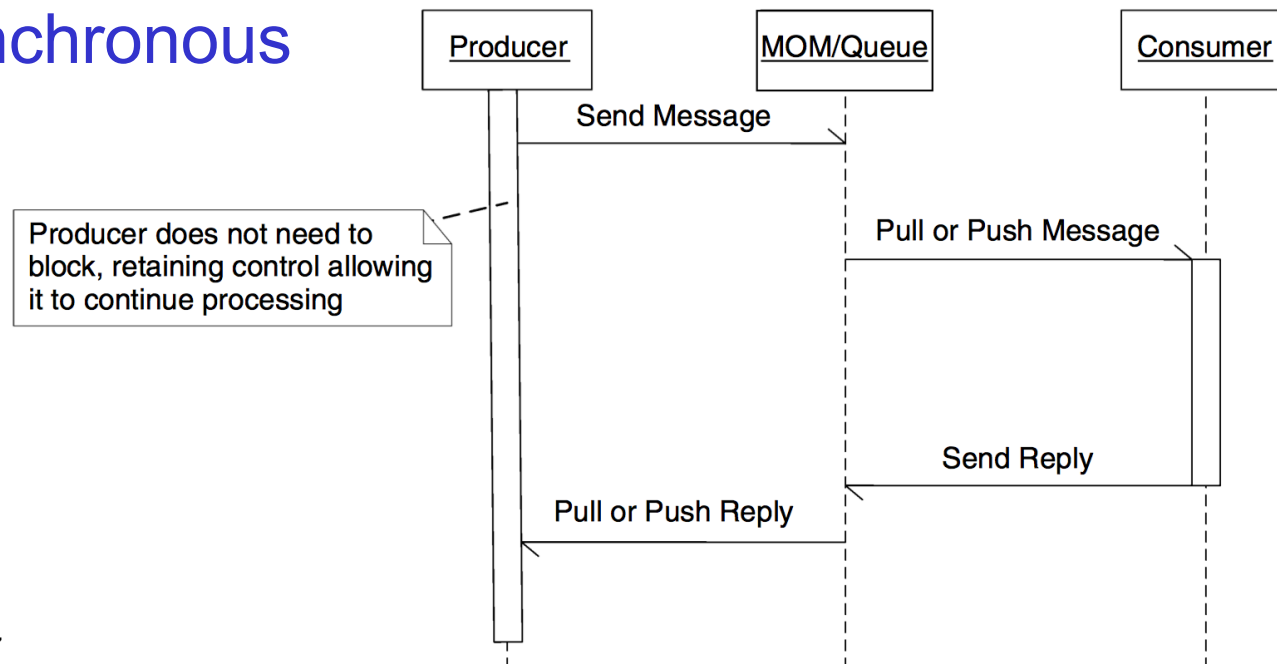
- **Space** (or **referential**) decoupling
  - Anonymous components: do not need to know each other in order to communicate and cooperate
- **Time** (or **temporal**) decoupling
  - Interacting components do not need to be present at the same time when communication occurs
- **Synchronization** decoupling
  - Interacting components do not need to wait each other and are not reciprocally blocked

# Synchronous vs. asynchronous interaction

## Synchronous



## Asynchronous



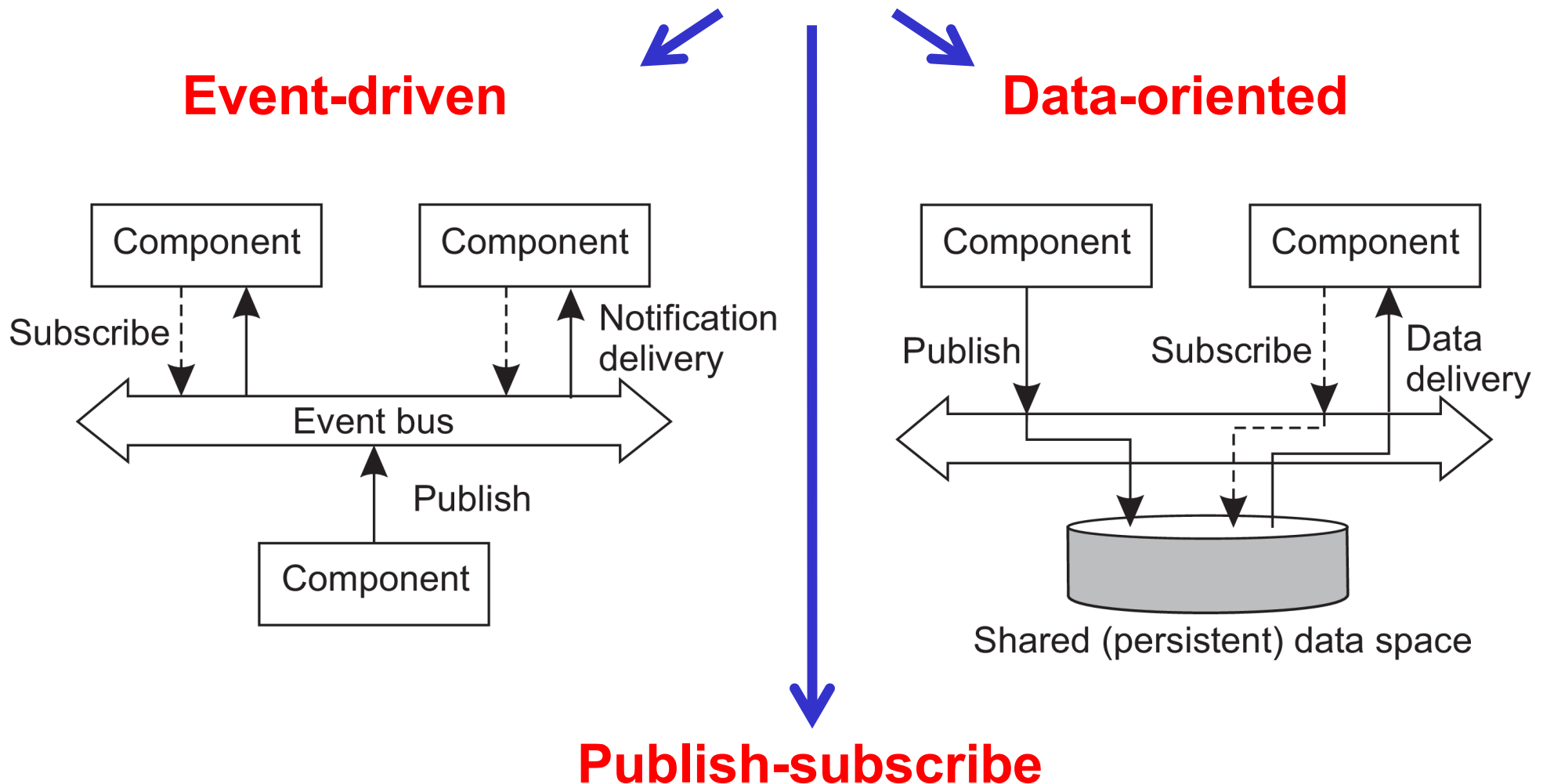
# Decoupling: pros and cons

---

- ✓ Thanks to decoupling, DS can be flexible while dealing with changes and provide more dependable and elastic services
  - Space decoupling: components can be replaced, updated, replicated or migrated
  - Time decoupling: allows to manage volatility (senders and receivers can come and go)
  - Synchronization decoupling: no blocking
  
- ✗ Indirection can add performance overhead

# Architectural style evolution

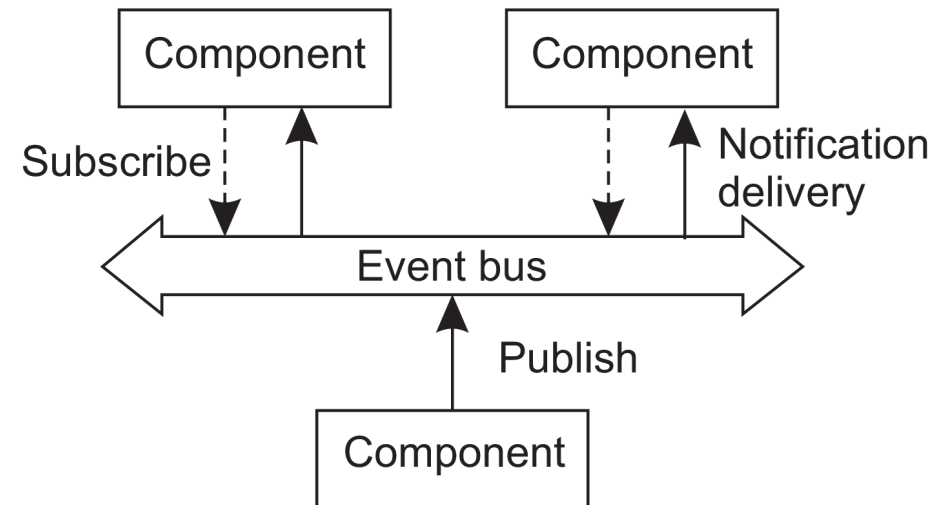
- Introducing decoupling, alternative architectural styles where components **communicate indirectly**



# Event-driven style

---

- Components communicate through an event bus
  - **Event**: state change (e.g., temperature, door opening)
- Components
  - Publish events
  - Subscribe to events they are interested in being notified
  - Receive notifications about events
- Communication
  - Anonymous
  - Based on message exchange
  - Asynchronous
  - Multicast



- Example: Java Swing

Which decoupling?

# Data-oriented style

- Communication among components happens through **shared data space: passive**, sometimes (pro)**active**

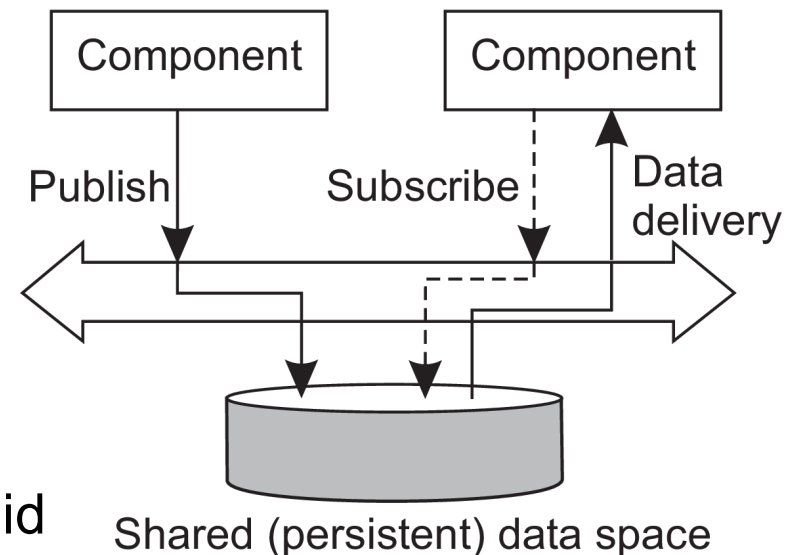
- Data added to or removed from shared space

- Shared data space API

- write, take, read and variants (takeIfExists, readIfExists)
- If active space: notify or push (avoid polling)
- Concurrency control

- Examples of shared data spaces

- GigaSpaces <https://tinyurl.com/bdtc5yk6>
- TIBCO ActiveSpaces <https://bit.ly/4hxCCrH>

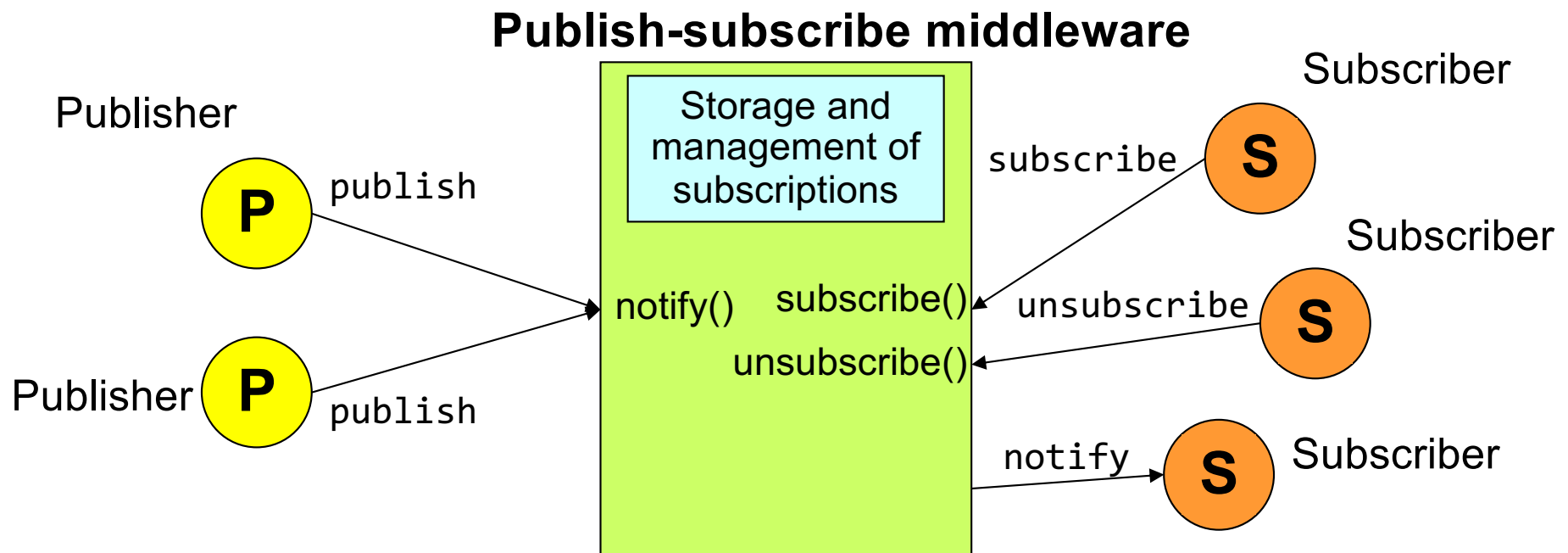


Which decoupling?

How to implement shared space?

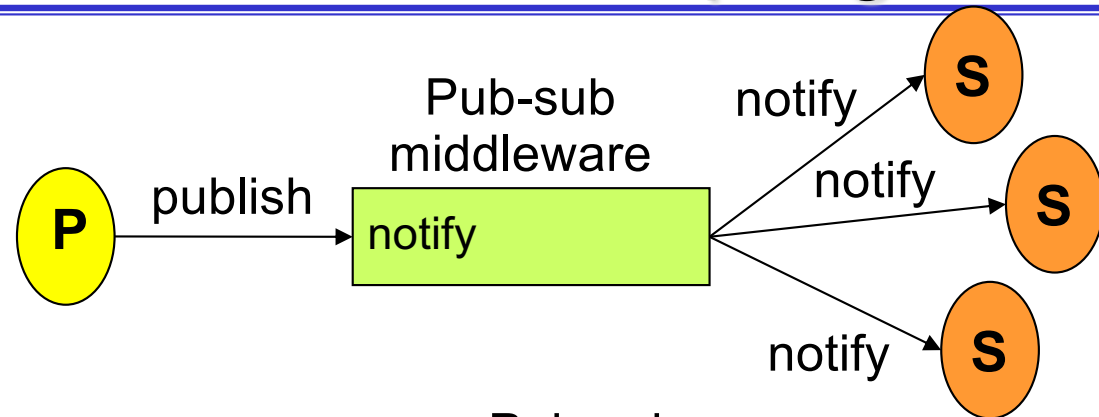
# Publish-subscribe style

- **Publishers** (aka **producers**) generate events (**publish**) and are not interested in their delivery to **subscribers** (aka **consumers**)
- **Consumers** register as interested to events (**subscribe**) and are notified (**notify**) of their occurrence
- Full decoupling among components

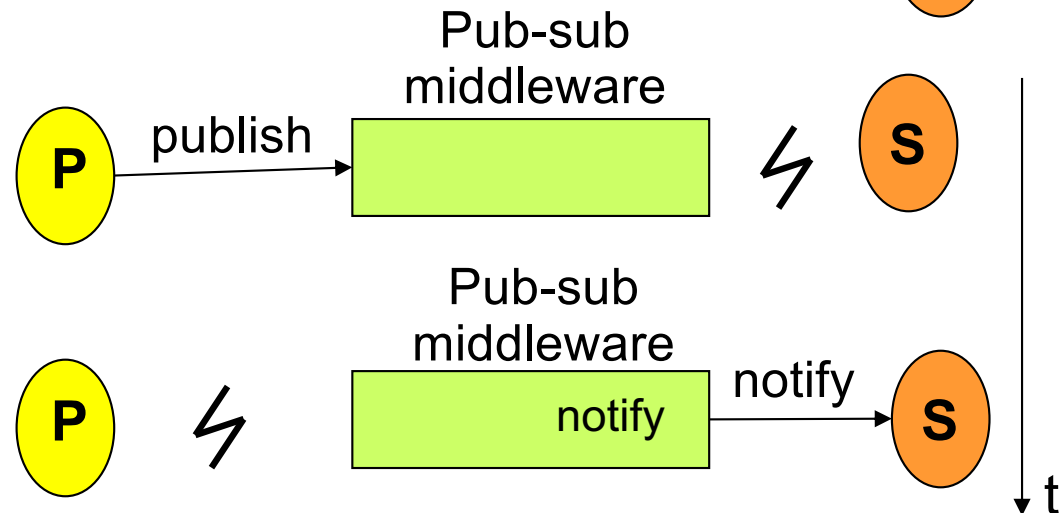


# Publish-subscribe and decoupling

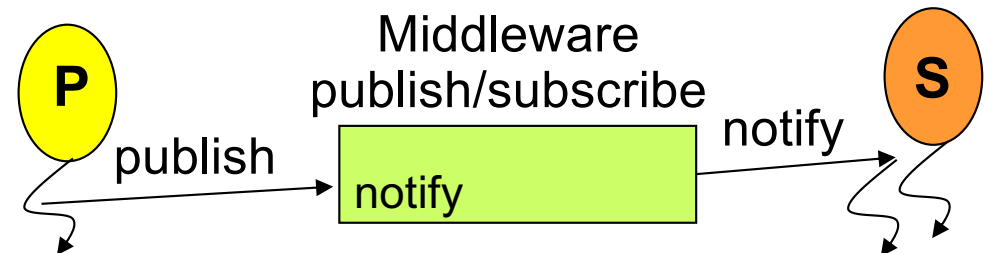
*Space* decoupling



*Time* decoupling



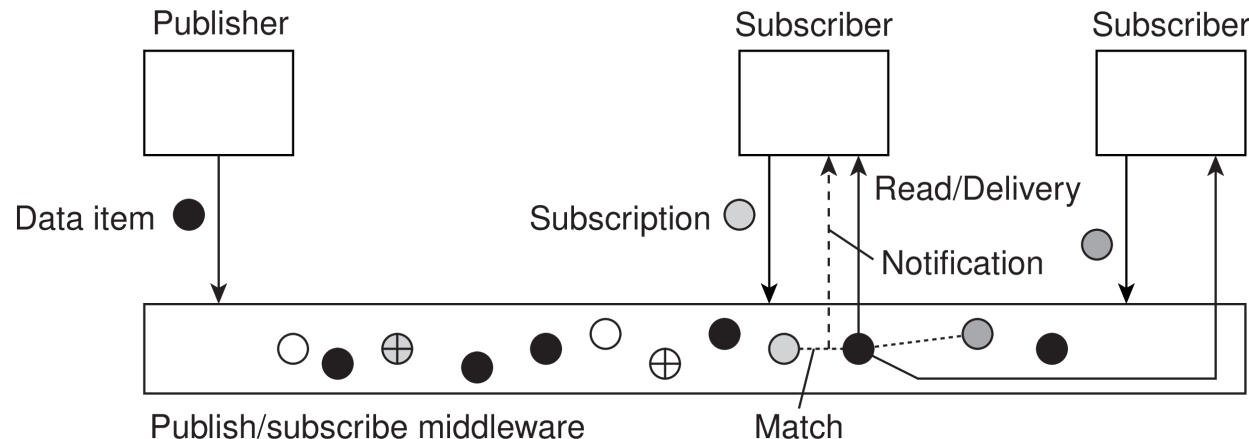
*Synchronization* decoupling



Eugster et al., The many faces of publish/subscribe, *ACM Comput. Surv.*, 2003

# Publish-subscribe: subscription models

- Issue: how to match events
  - Assume events are described by (attribute,value) pairs



- **Topic-based** subscription
  - Specify an “attribute = value” series (e.g., type=news)
  - ✗ Limited expressiveness, only exact match
- **Content-based** subscription
  - Specify a series of filter (e.g., “attribute ∈ range”)
  - Subscribers define complex conditions (e.g., price > 100 AND category = electronics)
  - ✗ Scalability challenges, why?

# Choosing an architectural style

---

- **No single solution**: the same problem can be solved using different architectural styles
- Driven by **non-functional requirements**:
  - Cost: resource usage and development/operational effort
  - Scalability and elasticity: ability to scale-out/in and adapt
  - Performance: response time, latency, ...
  - Fault tolerance: resilience and graceful degradation
  - Maintainability: ease of extending with new components
  - Usability and reusability: ease of configuration, usage, and component reuse

# Choosing an architectural style

---

- Why choose just one style? Mix them!
- Real-world systems combine multiple architectural styles tailored to different parts of the DS
- Example:
  - REST for client-facing APIs
  - + Event-driven or pub/sub for asynchronous background workflows
  - + Synchronous RPC for internal communication between microservices
  - + Data-driven for analytics and logging

# System architecture of DS

---

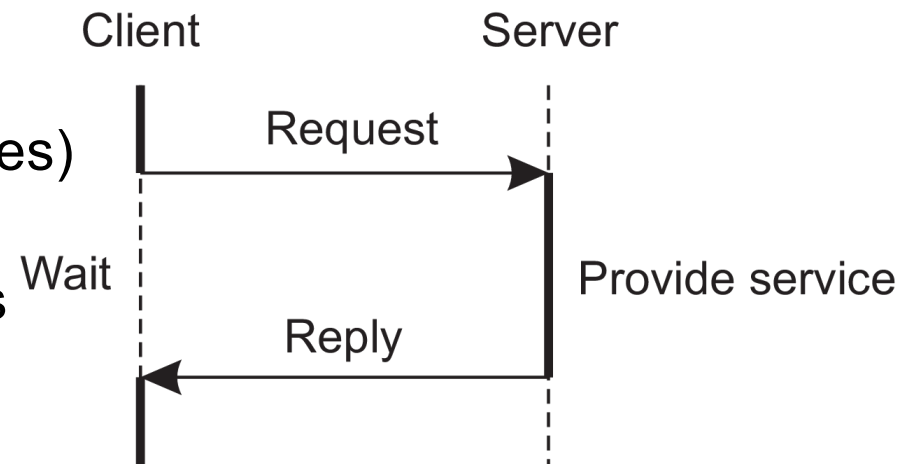
- **Runtime instantiation** of DS software architecture
  - Which components?
  - How do they interact with each other?
  - Where to deploy them?
- Types of system architectures
  - Centralized architectures
  - Decentralized architectures
  - Hybrid architectures

# Centralized system architectures

---

- Basic **client-server** model

- Two parties: servers (offer services) and clients (consume services)
- Can reside on different machines
- E.g., Web clients and servers



- **Request/reply** model
- Communication
  - based on message exchange
  - often *synchronous* and *blocking*
- Strong coupling
  - E.g., coexistence of interacting entities

# Multi-tier client-server architectures

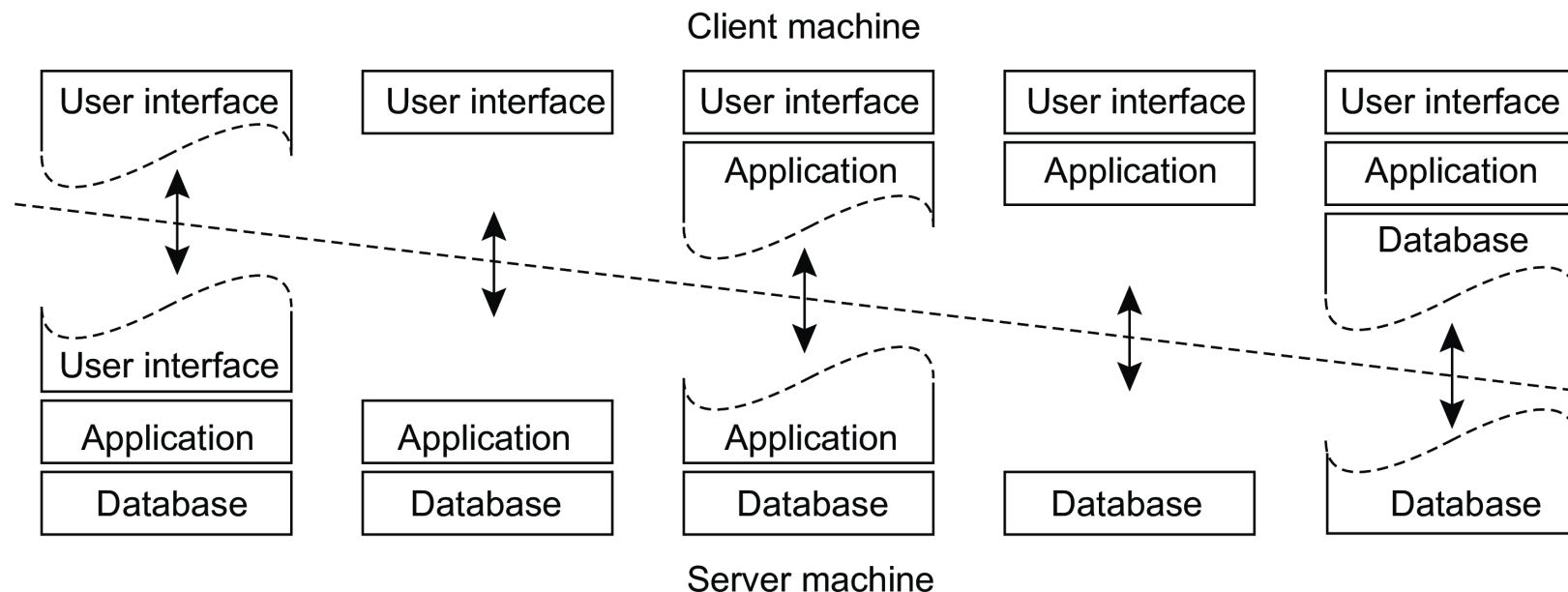
---

- How to map **logical levels** (*layers*) into **physical levels** (*tiers*)?
  - Two-tier architectures
  - Three-tier architectures
  - N-tier architectures

# Multi-tier client-server architectures

- Different configurations, depending on distribution of:
  1. **Presentation** layer
  2. **Logic** (aka application, business, processing) layer
  3. **Data** layer

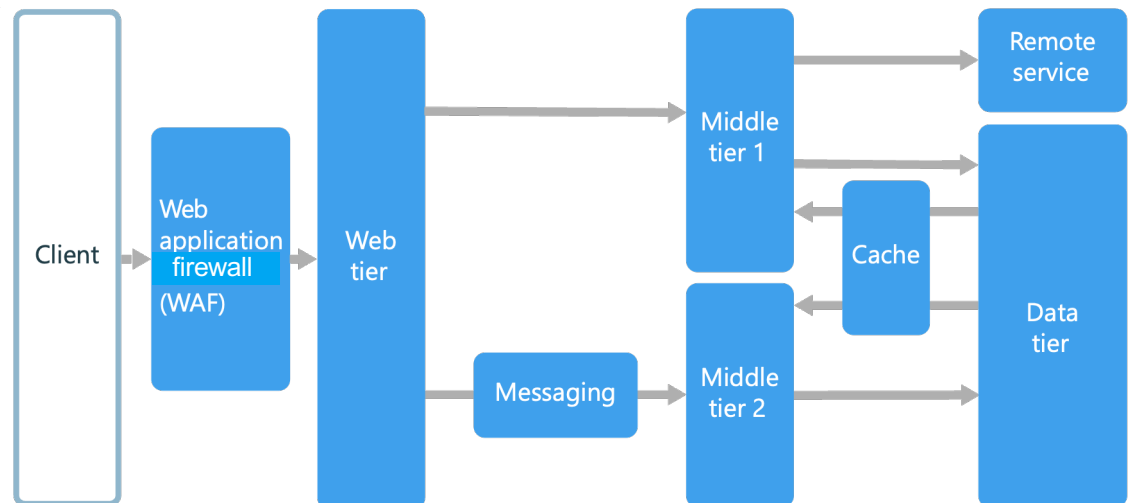
- Example: two-tier configurations



# Multi-tier client-server architectures

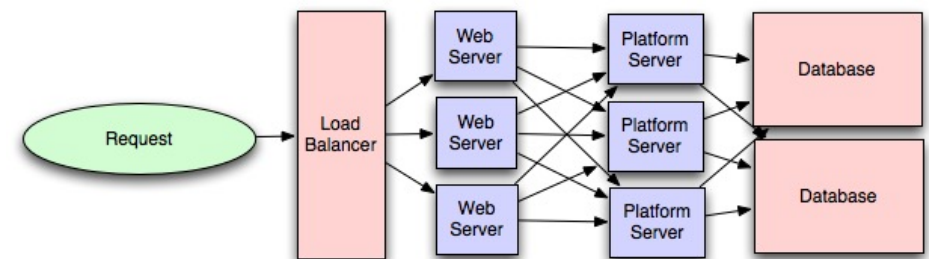
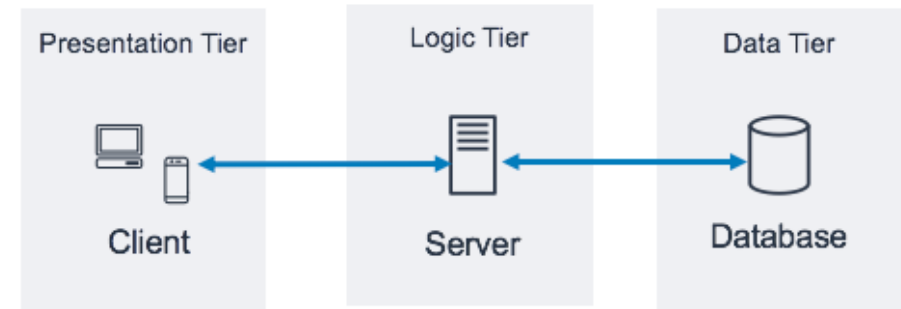
- From 3-tier to N-tier architectures
  - Adding components, e.g., load balancer, caching layers
- Architecture expands into **N logical layers** physically distributed across **N physical tiers**
- Pros and cons
  - ✓ Higher scalability and modularity: layers separate responsibilities and manage dependencies
  - ✗ Increased architectural complexity, operational overhead, and network latency

WAF: layer-7 security system that filters web traffic to protect applications from attacks



# From multi-tier architectures to...

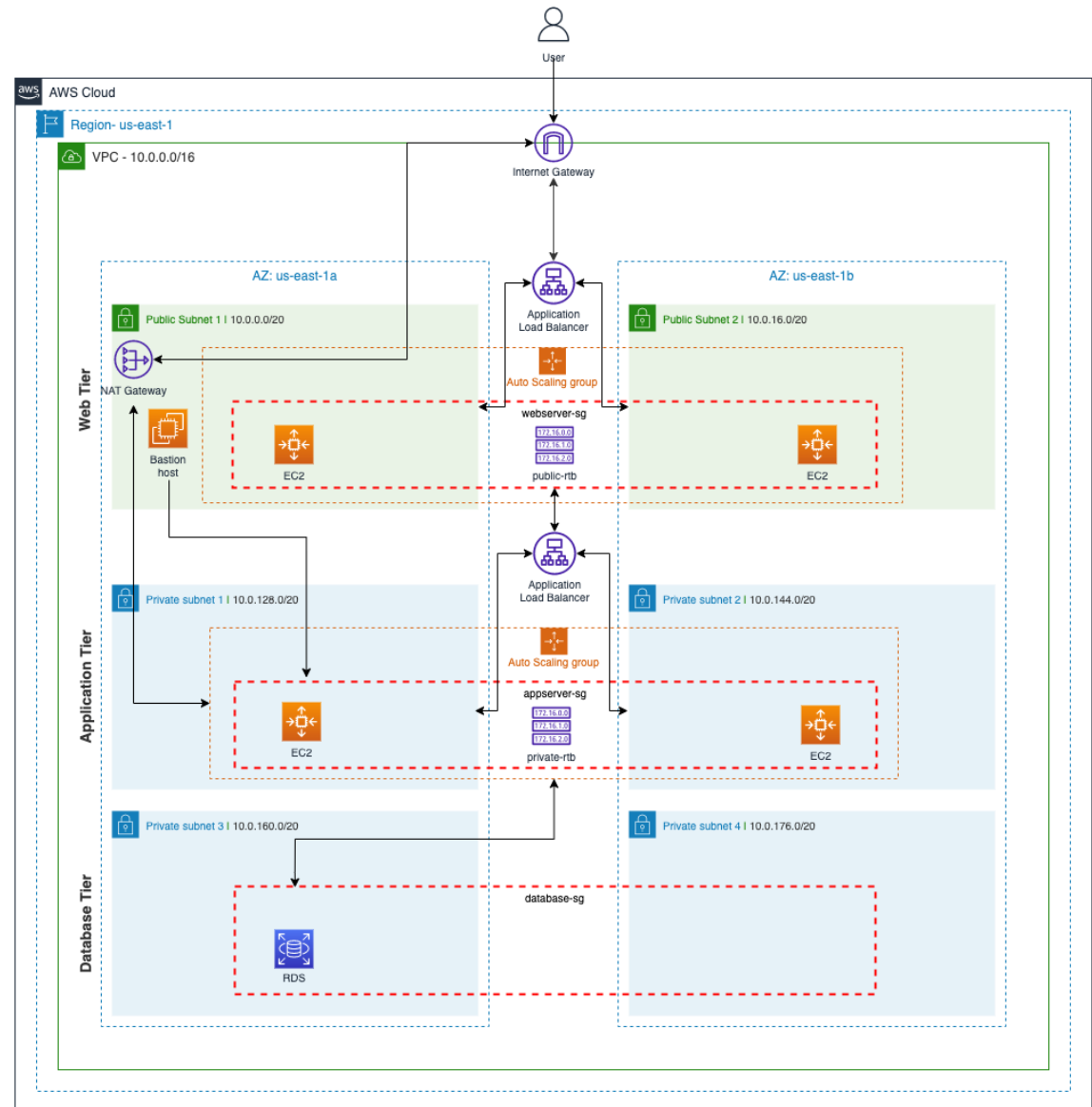
- **Vertical** distribution
  - 3 logical layers, where each layer runs on a different physical server
- **Horizontal** distribution
  - Distribute each layer on multiple servers
  - Balance load among multiple servers through a **load balancer**
  - E.g.,: distributed Web cluster



# Example: Web application in AWS

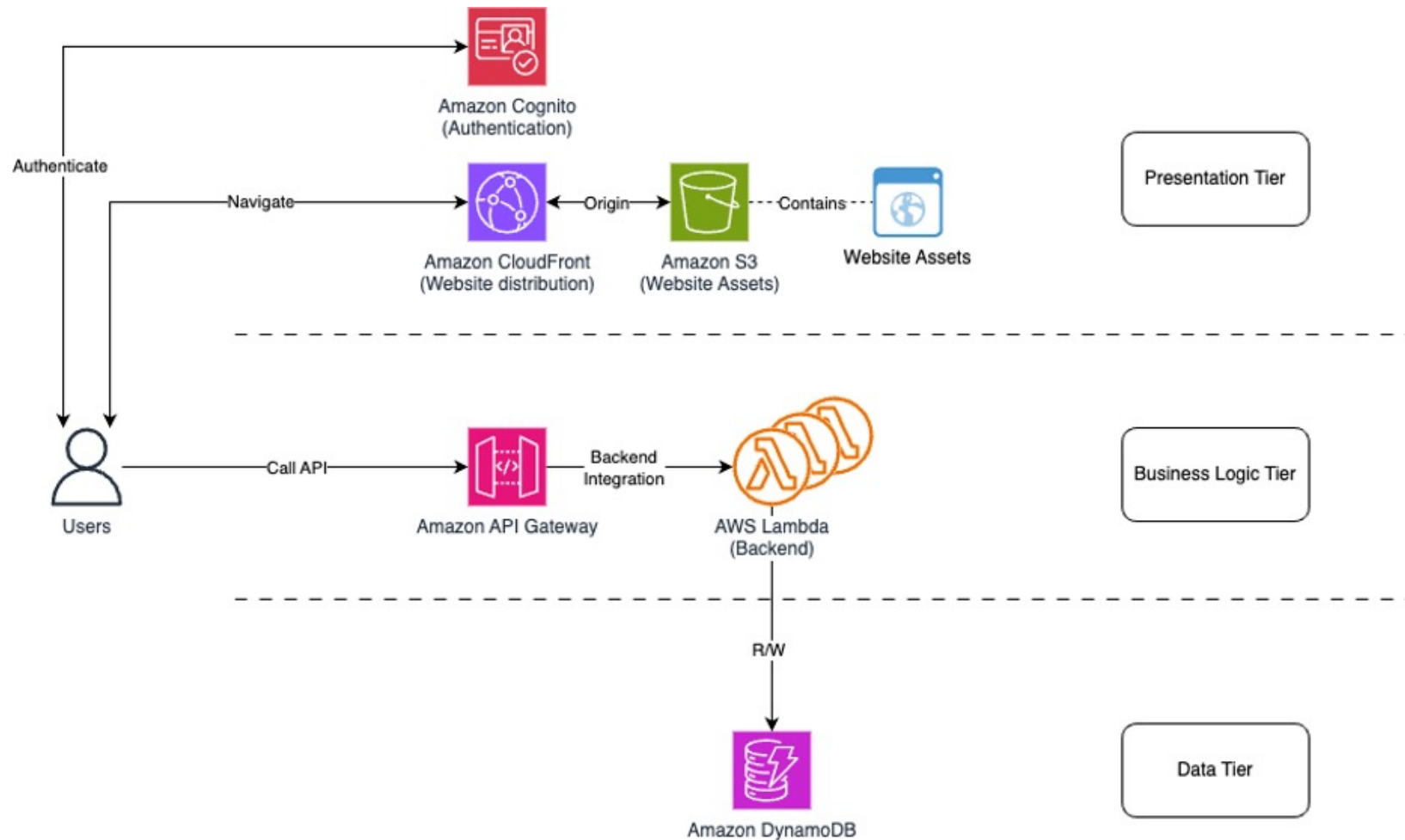
- Web application with horizontal distribution in AWS
  - **Elasticity**: tiers can scale out/in
  - **High availability**: replication in different availability zones
  - **Security**: tiers communicate with private IP

<https://bit.ly/4y4oQBI>



# Example: 3-tier serverless architecture

- AWS Lambda has built-in elastic scalability



<https://docs.aws.amazon.com/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/three-tier-architecture-overview.html>

# Decentralized system architectures

---

- **Peer-to-peer (P2P)** systems
- P2P: **decentralized** systems and applications using **distributed resources** to perform key functions
  - “P2P is a class of applications that takes advantage of resources available at the edges of the Internet” (Shirny, 2000)
- Leverage **shared resources**: files, storage, compute power, bandwidth
  - Peers both give and receive resources

# P2P systems: key characteristics

---

- Peers are roughly **symmetric** in roles, privileges and responsibilities
  - Peers act as both clients and servers (*servents*)
  - **Autonomous** nodes located at **network edge**
  - **Super-peer** may exist with enhanced roles
- No centralized control
- Highly distributed
  - Can scale to hundreds of thousands of nodes
- Highly dynamic
  - Nodes can **join** or **leave** at any time (*churn*)
- Redundancy of information
  - To ensure resilience and availability

# P2P: use cases and examples

---

- Content sharing and storage
  - File sharing: Kademlia, BitTorrent, WebTorrent
  - Video sharing: PeerTube
  - File storage: Freenet, IPFS
- Computing resource sharing
  - BOINC (sw platform for computing using volunteered resources), Folding@home (protein folding)
- Communication
  - e.g., jami, Matrix (+ Element), XMPP
- Blockchain and Web3
  - Bitcoin, Ethereum, Optimism, etc.: decentralized consensus and data sharing
  - libp2p networking library <https://libp2p.io>
- Decentralized AI training and inference
  - Exo, Bittensor

# P2P: challenges

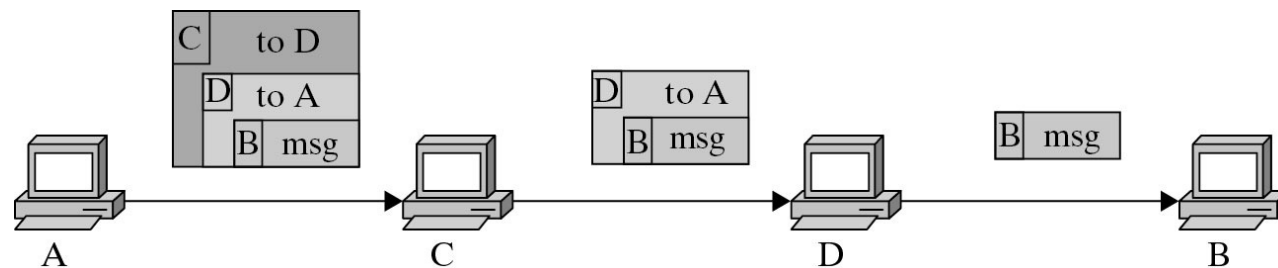
---

- Heterogeneity in peer resources
  - Peers differ in hardware, software, and network capabilities
- Scalability
  - Scale affects routing efficiency and bandwidth usage
- Location and interoperability
  - Data locality, network proximity, and cross-platform compatibility
- Fault tolerance and churn resilience
  - Peers join, leave, or fail randomly
  - Require robust failure recovery and data redundancy
- Performance
  - Routing efficiency, load balancing, self-organization
- Free-riding avoidance
  - *Free-riders* consume resources but do not contribute
  - Requires incentive mechanisms to promote fairness

# P2P: challenges

---

- Anonymity and privacy
  - Ensuring private communication
  - *Onion routing* (e.g., Tor) for anonymous message passing



- Trust and reputation management
  - Peers often do not know each other
  - Requires mechanisms to evaluate trustworthiness and prevent abuse
- Network threats and attacks
  - Vulnerable to sybil attacks, DDoS, malicious nodes
  - Require defense mechanisms like verification, redundancy, or rate-limiting

# Main tasks of a P2P node

---

- We consider **file sharing** as use case

## 1. Bootstrap

- How a new peer discovers others in the network and join the network
- Solutions: static configuration, cached peer list, or well-known nodes

## 2. Resource lookup

- How to locate resources
- Our **focus**

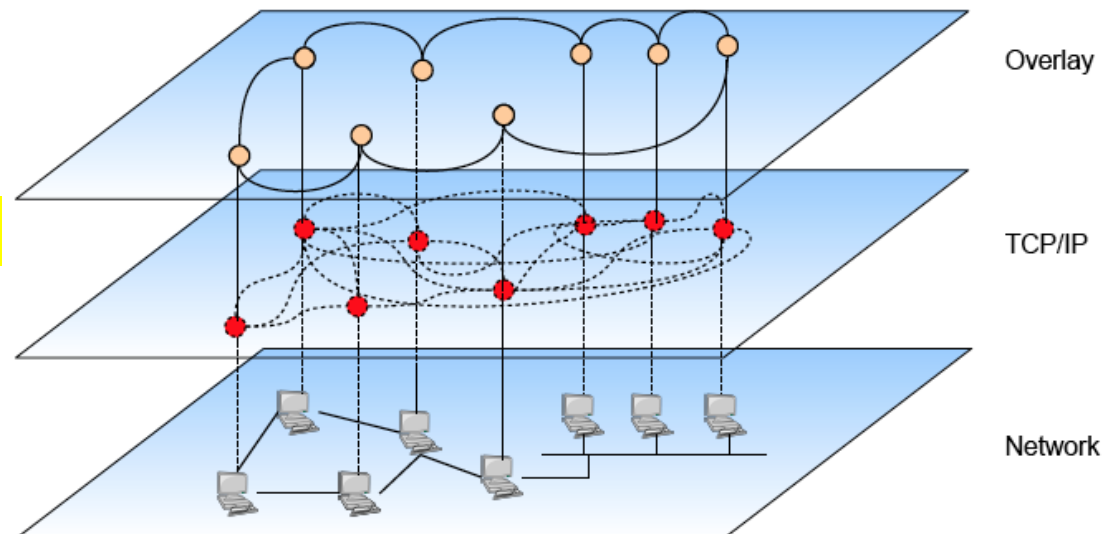
## 3. Resource retrieval

- How to download or access the located resource
- Solution: direct peer-to-peer data transfer, can be single-source or multi-source (different chunks simultaneously from multiple different peers)

# P2P as overlay networks

- P2P networks are often referred to as overlay networks
- **Overlay network**: **logical** network connecting peers built on top of physical IP network
  - Peers are connected by logical links, which may not match physical connections
  - Provides a resource location service by means of **application-level routing**

Application-level abstraction



# Overlay routing

---

- Basic idea:
  - Find a path to reach a resource in P2P system
- Unlike traditional routing
  - The target is a resource (file, CPU cycles, storage blocks), not a node address
- Our focus: **efficient routing** to locate resource
- Once located, resource retrieval is straightforward
  - Usually a direct peer-to-peer interaction (e.g., via HTTP)

# Overlay network operations

---

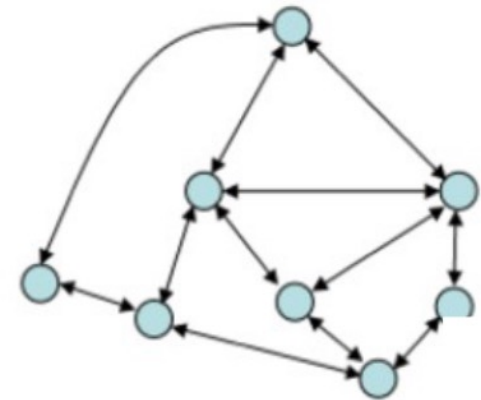
- Beyond routing, overlays enable:
  - Identifying resources and nodes
  - Inserting and deleting resources
  - Adding and removing nodes
- Resource identification
  - Use a Globally Unique Identifier (GUID)
  - Generated by applying a cryptographic hash function (e.g., SHA-1, SHA-256) to resource state/data (e.g., file name and content)
- Peer identification
  - Peer GUID also generated by applying a cryptographic hash function to peer information (e.g., IP address and port)

# P2P overlay classification

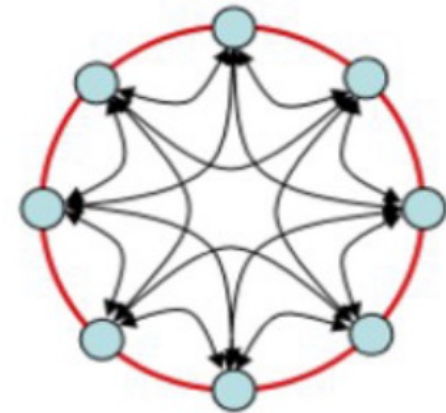
---

- How to manage resources and nodes?
  - Depends on overlay network's type

## Unstructured overlay networks



## Structured overlay networks



# Unstructured overlay network

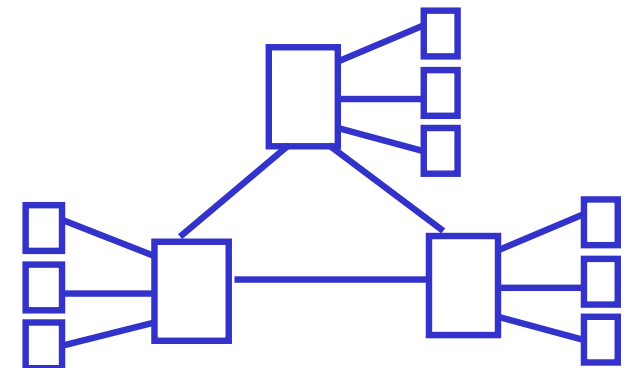
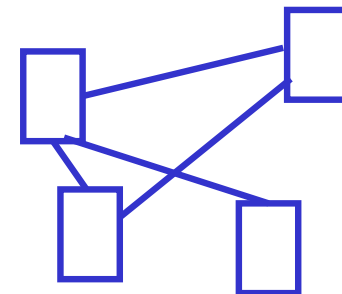
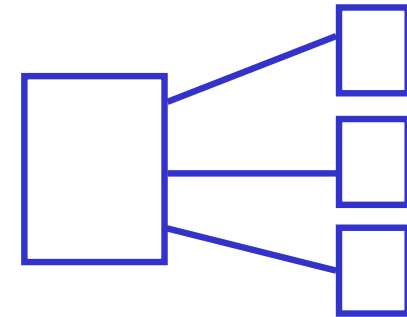
---

- Overlay network
  - Built on **random graphs** **without structure by design**
  - Peers connect arbitrarily using simple and local rules
  - New nodes join by contacting a set of neighbors
- Goals
  - No global control over resource placement on nodes
  - Designed for high network churn
- Examples: Gnutella, Bitcoin
- Pros and cons:
  - ✓ Low maintenance: easy insertion/deletion of nodes and resources
  - ✓ Highly resilient
  - ✗ Inefficient lookup (high cost and traffic overhead): searching for resources is complicated by the lack of structure

# Unstructured overlay network: routing

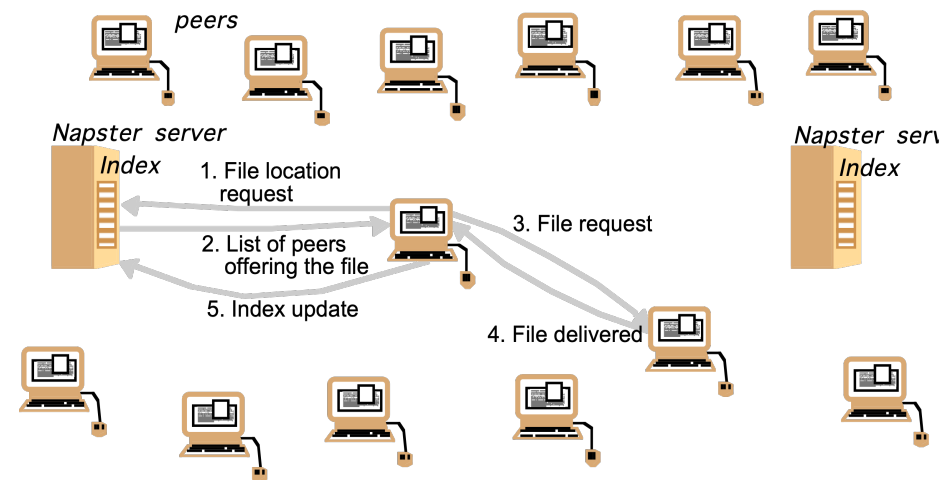
---

- We classify unstructured overlays by **distribution of peer-resource index** (*directory*)
- **Centralized** unstructured overlay: central directory (e.g., Napster)
- **Decentralized** unstructured overlay: distributed directory (e.g., Gnutella)
- **Hybrid** unstructured overlay: semi-centralized directory
  - Routing limited to *super-peers*



# Centralized unstructured overlay

- One or few nodes (**directory server**) maintain a global index of resources
- Peers query directory to locate resources



- ✓ Simple
- ✗ Single directory server: performance and scalability bottleneck
- ✗ Single point of failure (SPOF) for technical and legal reasons
- ✗ Expensive to manage due to centralized control

# Decentralized unstructured overlay

---

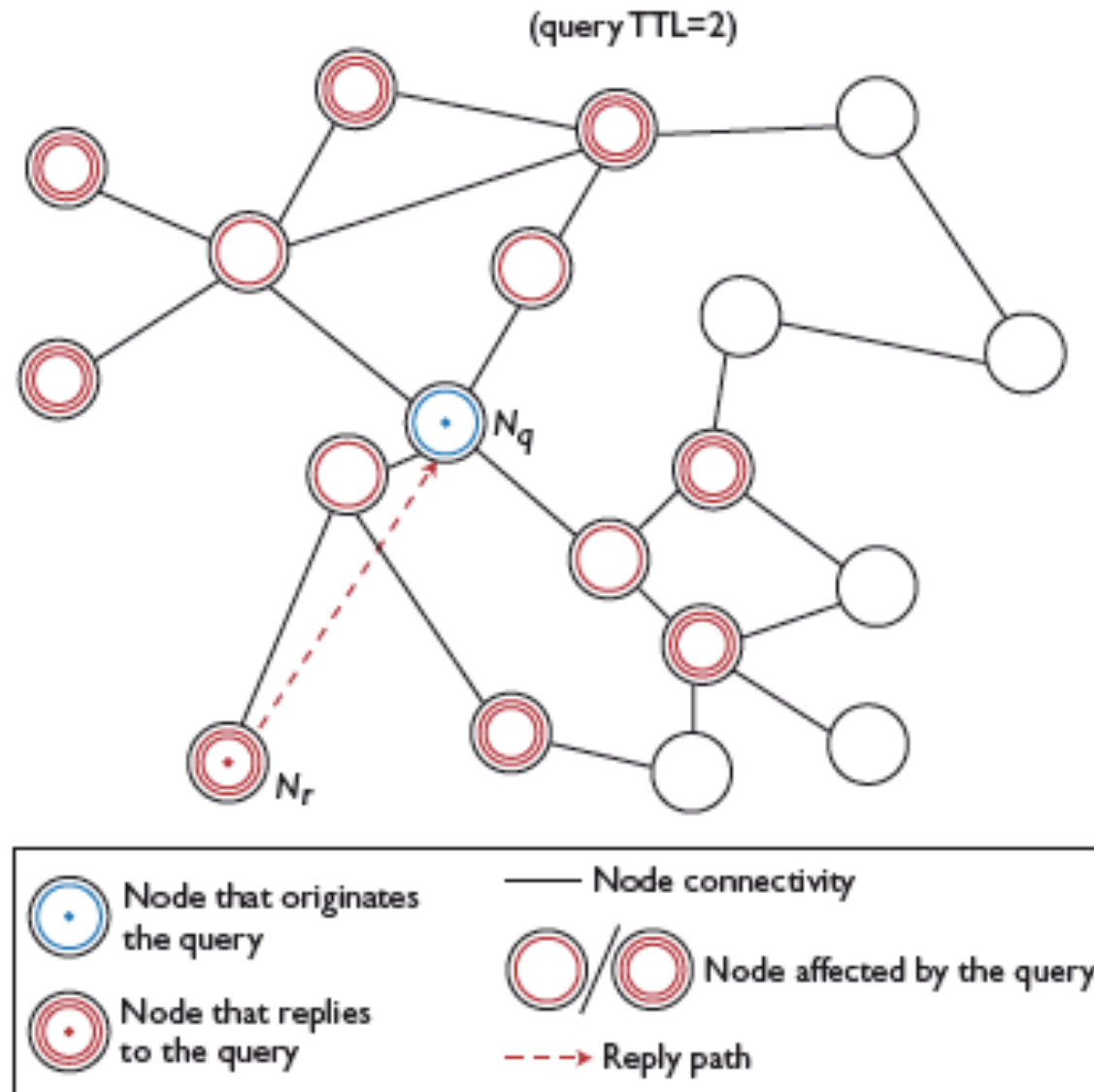
- Decentralized resource lookup
- How to find resources without a directory?
  - **Query flooding**: broadcast queries to neighbors until resource is found or TTL expires
  - **Random walk**: send queries along random paths to reduce message overhead
  - **Gossiping** (covered in upcoming lesson): probabilistic message spreading among peers

# Query flooding

---

- Originator sends query to its **neighbors**
- Each peer
  - Responds if it has the resource
  - Otherwise, **forwards** query to its neighbors (except sender)
- Optimization #1: limit search range
  - Use **Time-to-Live (TTL)**, decreasing it by 1 at each hop
  - Stop forwarding when  $TTL=0$
- Optimization #2: avoid cycles
  - Assign **unique query ID** so to not process same query multiple times
- Lookup cost:  $O(N)$ , where  $N$  = number of nodes

# Query flooding: example



# Query flooding

---

- Response routing options
- Direct routing
  - Resource owner sends response **directly** to query originator
- Backward routing
  - Response travels back along the same path as the query
  - Uses query ID to retrace the path
  - Which pros?

# Query flooding: cons

---

- Communication overhead
  - Large number of messages
  - Even unsuccessful queries consume bandwidth
- High lookup cost:  $O(N)$ 
  - How to choose TTL?
- Security risks
  - Vulnerable to DoS attacks
  - Risk of black-hole nodes under network congestion
- False negatives
  - No guarantee that (all) nodes with resource are reached
- Topology mismatch
  - Overlay “neighbors” may be physically far apart
  - Leads to inefficient routing and increased latency

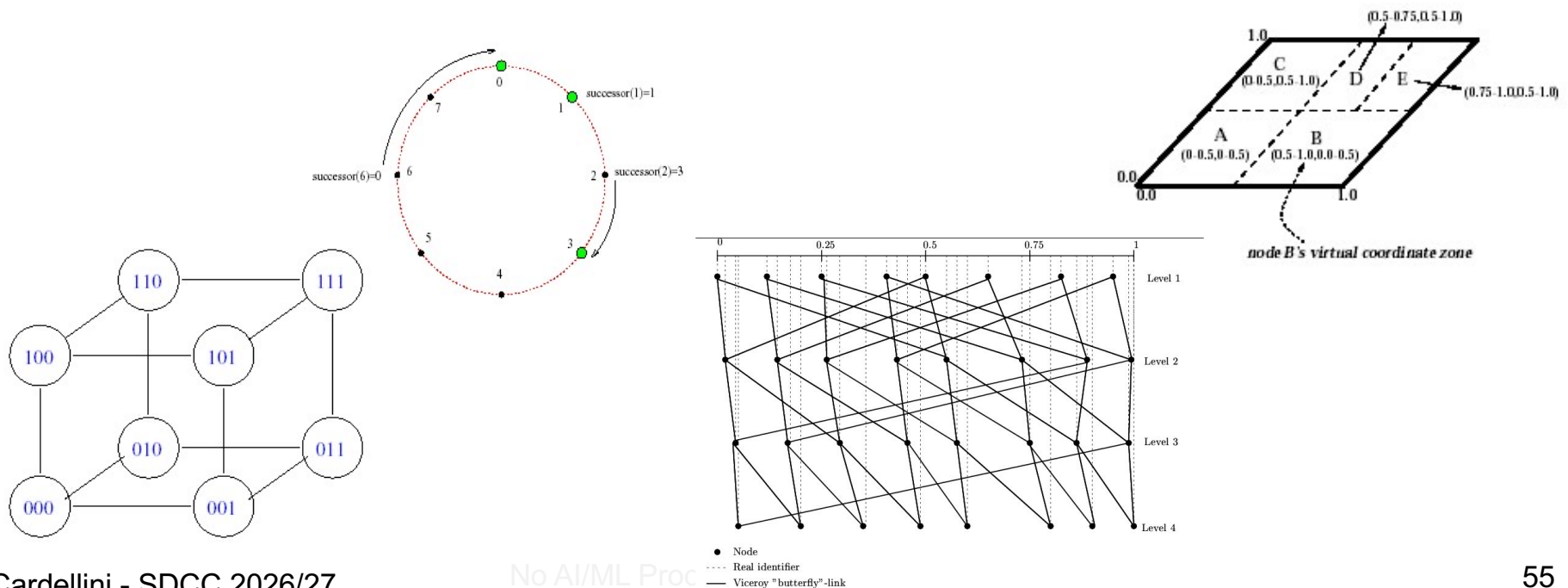
# Random walk

---

- The originator sends the query to **one random neighbor**
  - Each peer forwards the query to one of its neighbors, randomly chosen
  - Process continues until the resource is found
- Random walk vs. flooding
  - ✓ Lower message overhead
  - ✗ Longer lookup time
- Optimization:  **$k$ -random walks**
  - Originator starts  **$k$  independent random walks**
  - Sends the query to  $k$  randomly chosen neighbors
  - Each copy follows its own path
  - Balances lower traffic with faster discovery

# Structured overlay networks

- Lookup queries are forwarded using a well-defined network structure
- Each peer keeps track of a specific set of other peers
- Overlay network is **structured**
  - Constraints on how peers and resources are organized
  - Overlay **topology**: ring, tree, hypercube, grid, ...



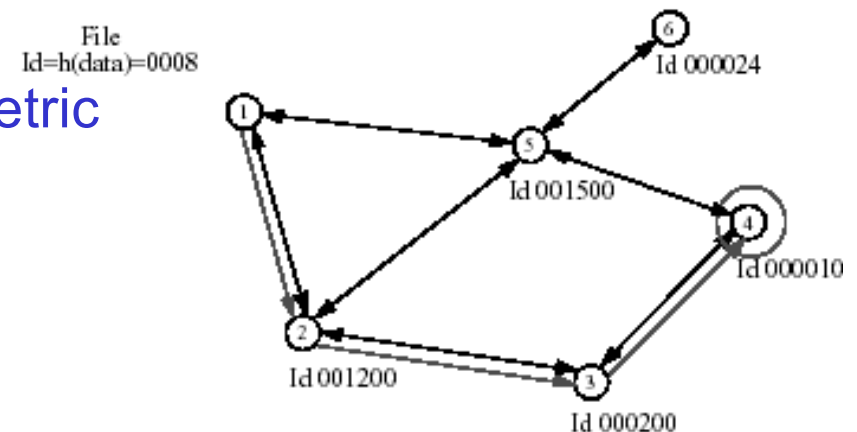
# Structured overlay networks

---

- Goals:
  - Scalability: lower lookup cost and reduced communication overhead compared to unstructured overlays
  - Efficient key-based resource lookup
    - Bounded lookup complexity (e.g., logarithmic)
  - Bounded complexity guarantees also for peer join/leave
- Trade-offs/cons:
  - Overlay topology must be maintained
  - Peer join and leave are more expensive

# Routing in structured overlays

- Basic concepts
  - Each peer is responsible for a **subset of resources** and knows a **subset of peers** according to the overlay structure
  - Each resource is assigned a **GUID**
  - Each peer is assigned a GUID
  - GUIDs are computed via a **secure hash function**
  - **Same large identifier space** used for peers and resources GUIDs
  - Lookup query is routed toward the peer whose GUID is **closest** to resource GUID
  - Closest: defined using a **distance metric**
- Routing is based on **Distributed Hash Table (DHT)**
  - A distributed key-value store spread across all peers



# Distributed Hash Table

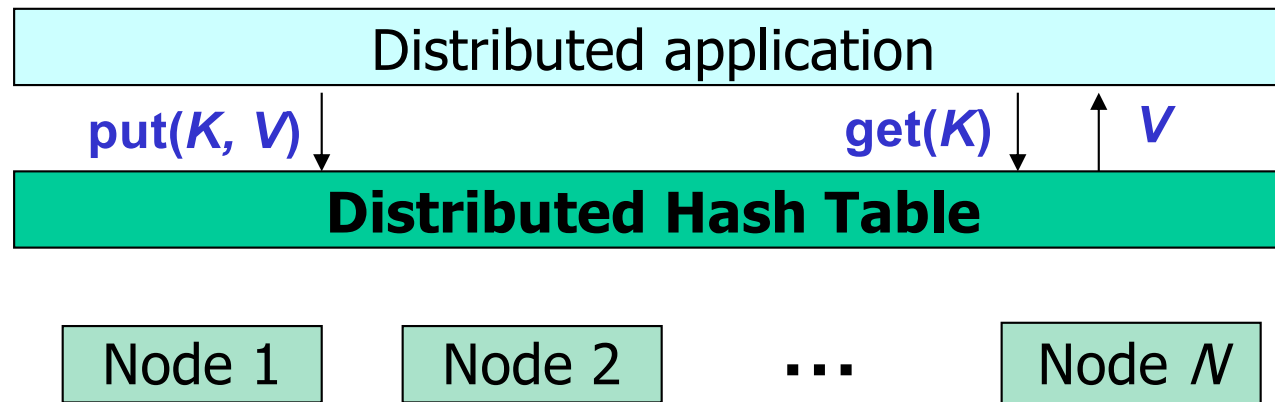
---

- Distributed abstraction of conventional hash table (HT) that maps keys to values
- Conventional HT
  - Stores (*key*, *value*) pairs in a table of size  $M$
  - Uses a hash function to map a key to an index  $0 \dots M-1$
  - Lookup time:  $O(1)$
  - Must handle collisions because multiple keys may hash to same value
- DHT
  - Conceptually similar to conventional HT: map resource key through hashing to determine the responsible peer (bucket)
  - DHT buckets are spread across peers
- Key challenge: how to efficiently map a resource key to the peer responsible for that key's bucket?

# DHT: API

---

- DHT stores key-value pairs (key  $K$ , value  $V$ )
  - $K$  = resource GUID,  $V$  = resource
- Common DHT API operations
  - $V = \text{get}(K)$ : retrieve  $V$  for  $K$  from the responsible node
  - $\text{put}(K, V)$ : store  $V$  at the node responsible for  $K$
  - $\text{remove}(K)$ : delete the reference to  $K$  and its associated  $V$



# Why might DHT design be hard?

---

- Decentralized
  - No central authority
- Scalable
  - Low network traffic overhead
- Efficient
  - Find items quickly (latency)
- Dynamic
  - Nodes fail or leave, new nodes join

# Designing a DHT

---

- Mapping resources and nodes in DHTs
  - Both resources and nodes are mapped to the **same identifier space** using a **hash function** (e.g., SHA-1, SHA-256)
  - GUIDs are composed of  $m$  bits (e.g.,  $m = 160, 256$ )
  - Hash function is applied to metadata/data of resources (name, timestamp, content, ...) and of nodes (IP address, port, ...)
- Resources are partitioned among nodes
  - Each node is assigned a **contiguous range** of keys and stores resources whose keys fall in its range
- Routing in DHT
  - Given  $K$ , map it to the node with GUID **“closest”** to  $K$
- Resource replication can be used to improve availability and provide fault tolerance
  - E.g., neighbor replication: replicate resource on nodes that are neighbors in the ID space

# Issues related to DHTs

---

- Avoid hotspots by evenly distributing key responsibility among peers
- Avoid remapping all keys if DHT size changes (i.e., when peers join or leave)
  - **Consistent hashing** addresses these issues
- DHTs only directly support **exact-match search**
  - Since each resource is identified only by its key, to lookup a resource we need to know its key
  - Exact-match queries (e.g., based on resource name) are easy to perform
  - More complex queries (e.g., wildcard, range queries) are difficult and expensive to support
  - We consider only exact-match queries

# P2P systems based on DHTs

---

- Characterized by **high scalability** with respect to system size (i.e.,  $N$ )
- Many proposals for DHT-based P2P systems, differ in:
  1. Definition of identifier space (and thus **network topology**)
  2. Selection of peers to communicate with (**distance metric**)
- Examples of DHT protocols:
  - **Chord** (MIT)
  - Pastry (Rice Univ., Microsoft)
  - Tapestry (Berkeley Univ.)
  - CAN (Berkeley Univ.)
  - **Kademlia** (NY Univ.)

# Why Chord?

---

- Elegant resource lookup algorithm for DHT
- Efficient:  $O(\log N)$  message per lookup
- Scalable:  $O(\log N)$  state per node
- Robust: survives massive failures
- Simple to analyze

Stoica et al., Chord: A scalable peer-to-peer lookup service for internet applications, ACM SIGCOMM 2001

# Chord

<https://github.com/sit/dht/wiki>

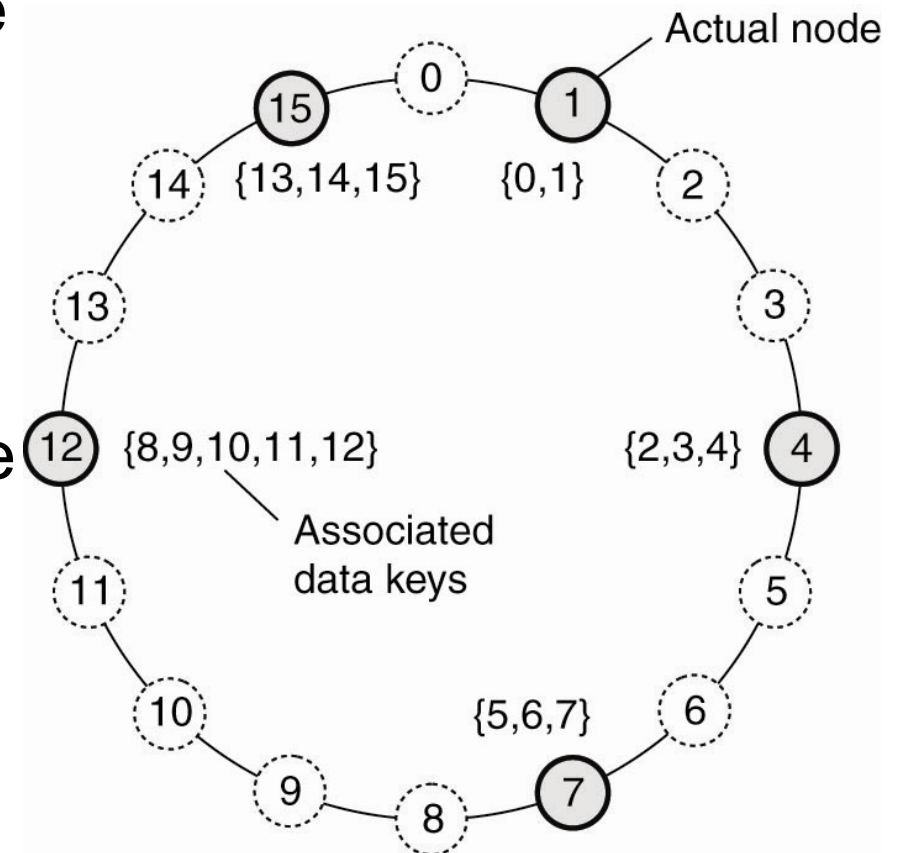
- Nodes (peers) and resources are mapped onto a **ring** using **consistent hashing**

- Each node is responsible for all keys between itself and its predecessor in counter-clockwise direction

- Resource with key  $K$  is managed by the node whose identifier is the smallest  $\text{id} \geq K$
- This node is called *succ*( $K$ ), **successor** of key  $K$

- E.g.,  $\text{succ}(1)=1$ ,  $\text{succ}(10)=12$

- Distance between IDs is measured as **linear difference** on the ring: if  $x \leq y$ ,  $d(x, y) = y - x$ ; else  $d(x, y) = 2^m - (x - y)$



# Consistent hashing

---

- Special hashing technique mapping both resources and nodes on the **same identifier space** (ring)
- Uses hash functions (e.g., SHA-1, MD5)
- Each node manages a **contiguous interval** of hash keys, not scattered keys
- Originally devised by Karger et al. (MIT, 1997) for Web distributed caching

Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the World Wide Web, STOC 1997

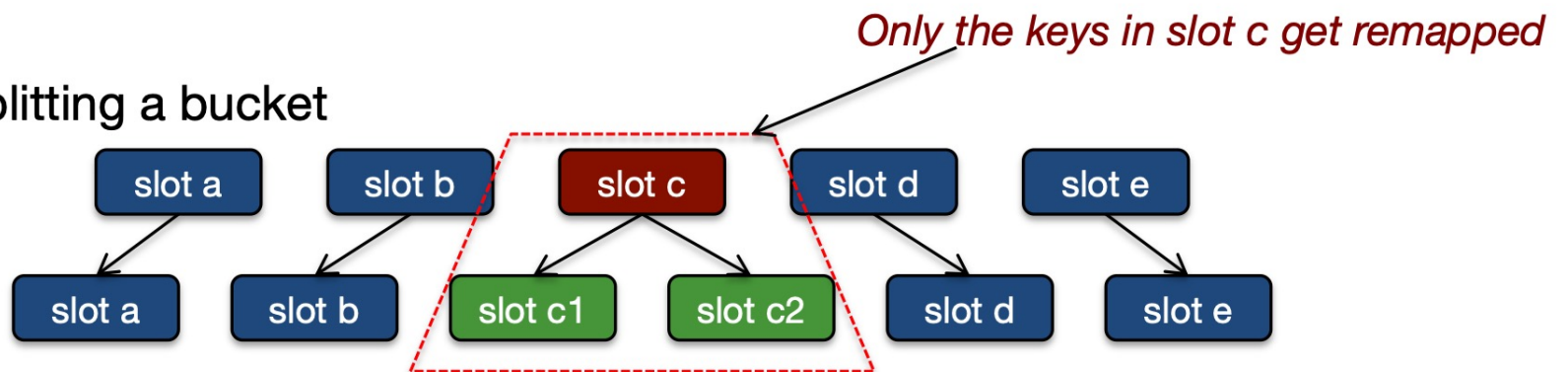
- Behind **Akamai** <https://www.akamai.com/>
- Widely adopted in real systems such as **Amazon Dynamo, Cassandra, Discord, Memcached**
- Details and Java implementation

<https://tom-e-white.com/2007/11/consistent-hashing.html>

# Chord: consistent hashing

- Consistent hashing is key to Chord robustness and performance
  1. When DHT **resizes** (nodes join or leave): minimal key remapping as most keys remain assigned to the same node
    - Practical impact: peers can join and leave with minimal disruption

Example: splitting a bucket



2. All nodes are assigned roughly the same number of keys:  
**load balancing** among peers

# Chord: towards routing

---

- The simplest approach: traverse the ring one node at a time
- Can we do better than  $O(N)$  lookup?
- Another simple approach for great performance
  - Every node knows all others
  - When a peer receives a query, it searches its table of nodes to find the node that owns that key
  - ✓ Lookup is  $O(1)$
  - ✗ Costly updates on join/leave
  - ✗ Not scalable for many peers (large tables)
- Chord uses a compromise to avoid large tables at each node: **finger table**
  - A partial list of nodes, progressively more distant

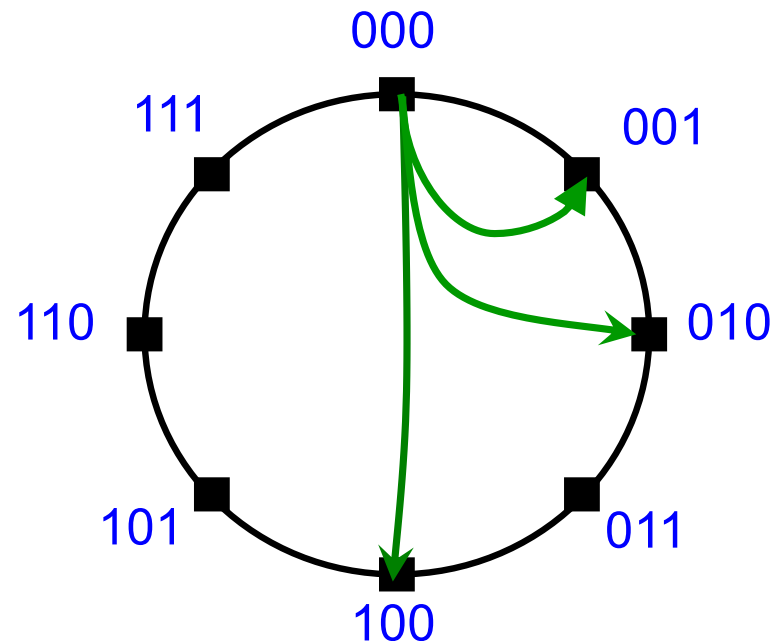
# Chord: finger table

- **Finger table** (FT): routing table of each node
  - FT has  **$m$  rows**, with  $m = \#$  GUID bits
  - If  $FT_p$  is the FT of node  $p$ , then  $FT_p[i] = \text{succ}(p + 2^{i-1}) \bmod 2^m$ ,  $1 \leq i \leq m$ 
    - $\text{succ}(p+1), \text{succ}(p+2), \text{succ}(p+4), \text{succ}(p+8), \text{succ}(p+16), \dots$
- Example with  $m=3$ 
  - Finger table of node 0?

$$FT_0[1] = 0 + 2^0 = 1$$

$$FT_0[2] = 0 + 2^1 = 2$$

$$FT_0[3] = 0 + 2^2 = 4$$



# Chord: FT's characteristics

---

- Each node stores information about only a small number of other nodes
  - Only  $m$  rows
- Each node knows more about nodes closely following it than about nodes farther away
- However, a node's FT generally does not contain enough information to directly determine the successor of any key
  - We need a routing algorithm to map each key  $K$  into  $\text{succ}(K)$

# Chord: routing algorithm

---

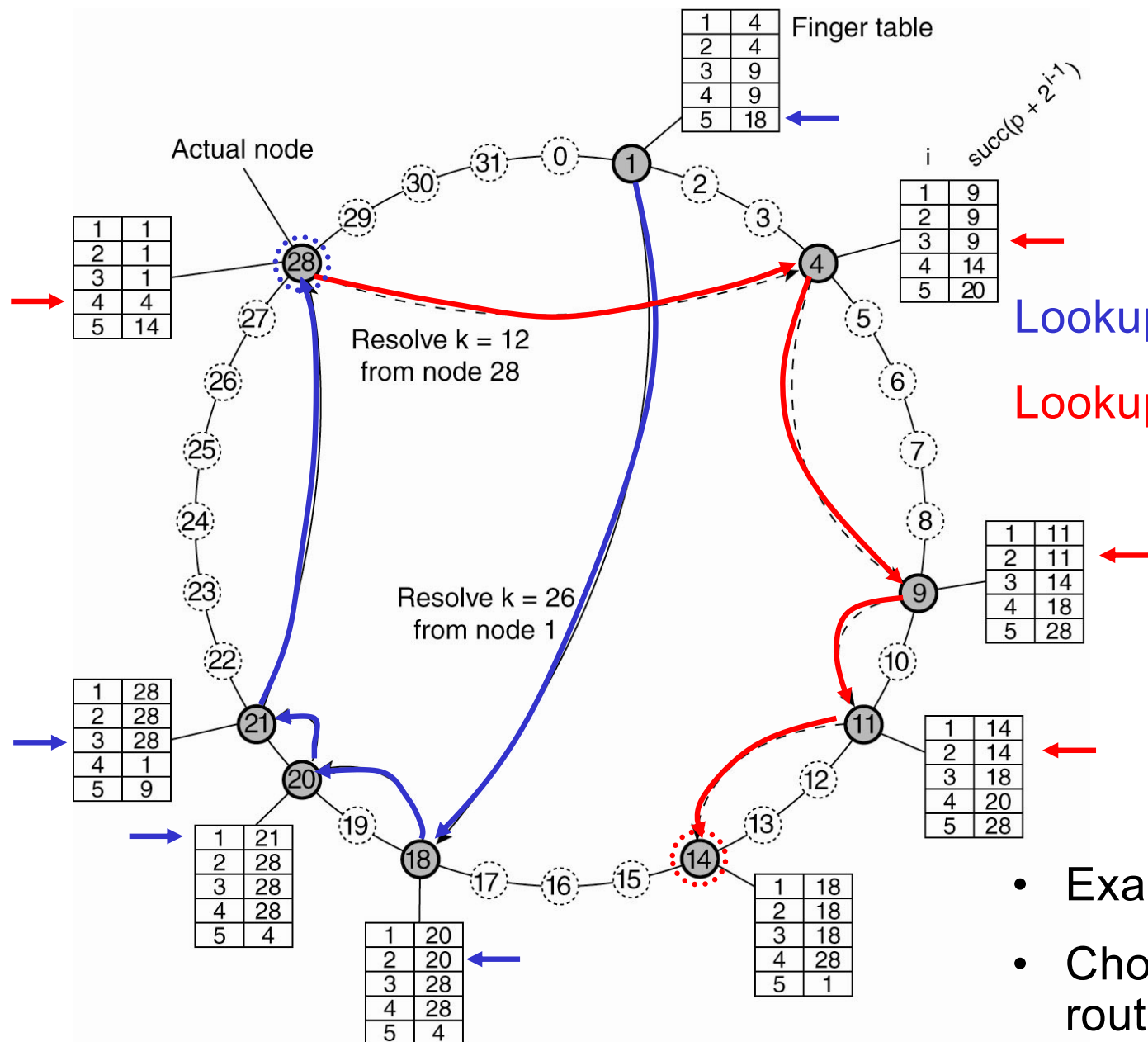
- How to map key  $K$  into  $\text{succ}(K)$  starting from node  $p$ 
  - If  $K$  belongs to the ring portion managed by  $p$ , lookup ends
  - If  $p < K \leq FT_p[1]$ ,  $p$  forwards the request to its successor
  - Otherwise,  $p$  forwards the request to node  $q$  with index  $j$  in  $FT_p$  by considering the clockwise ordering

$$FT_p[j] \leq K < FT_p[j+1]$$

$q$  is the farthest node from  $p$  whose key is less than or equal to  $K$

- Features
  - It quickly reaches the vicinity of the searched point, then make gradually smaller jumps
  - Lookup cost:  $O(\log N)$ , where  $N$  is the number of nodes ... not as cool as  $O(1)$  but way better than  $O(N)$

# Chord: routing algorithm example



In the example:

- $m=5$
- Keys from 0 to  $2^5-1$

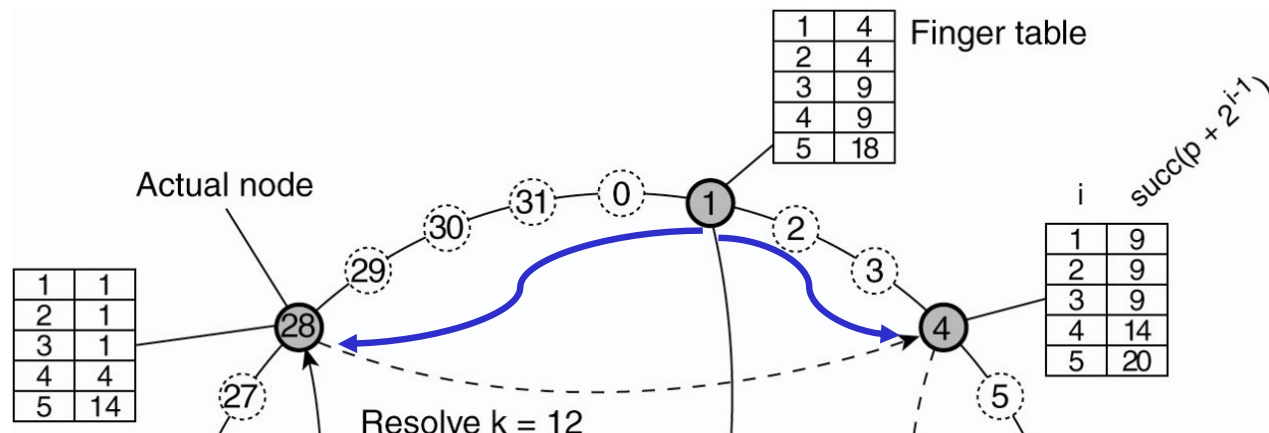
Lookup for key 26 from node 1

Lookup for key 12 from node 28

- Example of *recursive* routing
- Chord also supports *iterative* routing

# Chord: node join and leave

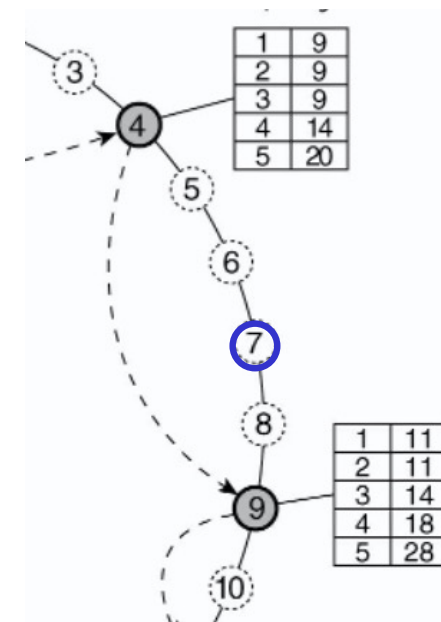
- In addition to maintaining a successor pointer, each node also keeps the pointer to its predecessor (i.e., linked list) to simplify ring maintenance operations
  - Predecessor of node  $p$  is the first node met in counter-clockwise direction starting at  $p-1$
  - When a node joins or leaves, **successor and predecessor pointers need to be updated**



# Chord: node join and leave

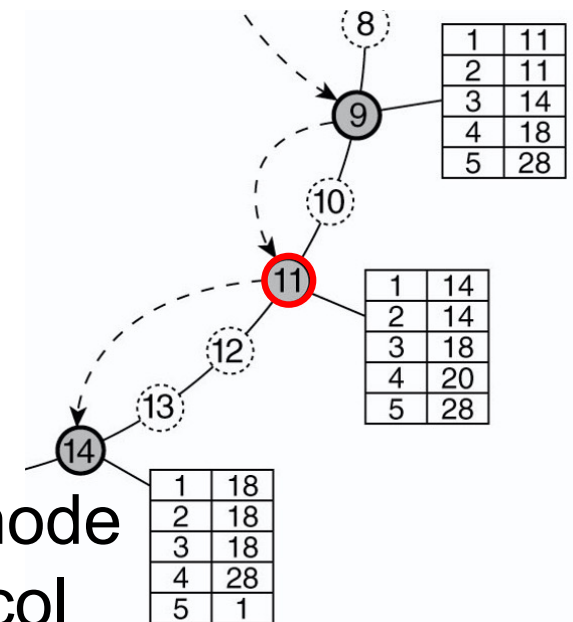
- When node  $p$  **joins** the Chord ring, it has to find its place:
  - Finds successor: asks another node to find its successor  $\text{succ}(p+1)$
  - Joins the ring, linking to its successor and informing its predecessor of its presence
  - Initializes its FT by looking for  $\text{succ}(p + 2^{i-1})$ ,  $2 \leq i \leq m$
  - Informs its predecessor to update its FT
  - Transfers the keys from its successor to itself

- Example: node 7 joins
  - Node 7's successor is node 9
  - Node 9's predecessor changes to node 7
  - Node 4's successor changes to node 7
  - Keys 5, 6, and 7 are transferred to node 7



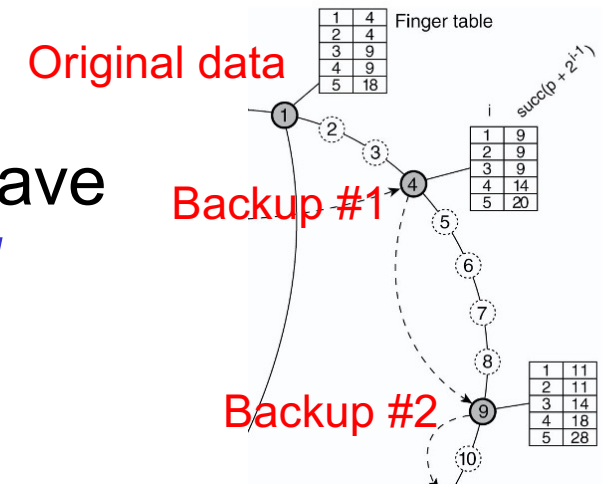
# Chord: node join and leave

- When node  $p$  **voluntary leaves** the Chord ring:
  - Transfers the keys it is responsible for to its successor
  - Updates its successor's predecessor pointer to  $p$ 's predecessor
  - Updates its predecessor's successor pointer to  $p$ 's successor
- Example: node 11 leaves
  - Keys 10 and 11 are transferred to node 14
  - Node 14's predecessor changes to node 9
  - Node 9's successor changes to node 14
- Join/leave operations require  $O(\log^2 N)$
- To keep the finger tables updated, each node periodically runs a **ring stabilization** protocol
- Nodes may leave the ring abruptly due to **failure**
  - Nodes must detect failed nodes, using timeouts or heartbeat mechanisms



# Chord: fault tolerance

- Nodes might **crash**
- Solution: **replicate**  $(K, V)$  pairs so to have  **$R$  replicas**, storing each replica at  **$R-1$  successor nodes** in the ring
- A node must be aware of multiple successors
  - Needs to find its successor's successor (or more)
  - Solution: each node maintains a **successor list**
- Upon recovery from crash, a node needs to:
  - Check with its successors for updates to keys it owns
  - Check with its predecessors for updates of replicated keys it stores as backups



# Chord: summing up

---

- Pros

- ✓ Simple and elegant
- ✓ Load balancing
  - Keys are evenly distributed among nodes
- ✓ Scalability
- ✓ Efficient lookup:  $O(\log N)$
- ✓ Robustness
  - Nodes periodically update their finger tables to reflect changes in the overlay, keeping the system robust

- Cons

- ✗ Physical network proximity not considered
- ✗ Expensive support for non-exact matching
- ✗ Incorrect Chord's original ring-maintenance protocol

Reasoning about identifier spaces: How to make Chord correct, IEEE TSE, 2017 <https://arxiv.org/pdf/1610.01140.pdf>